

Micro-services d'analyse des images rejetées en mammographie

BARBET, Mattéo

Branche : Informatique et Systèmes d'Information

Responsable pédagogique UTT :
M. SALEMBIER Pascal

Semestre : Printemps 2019

Résumé

Ce stage s'est déroulé dans l'entreprise General Electric Healthcare, à Buc (Yvelines), au sein de l'équipe de recherche et développement Software Apps Mammography.

L'objectif du stage était de développer un micro service connecté à l'application logicielle des mammographes. Celui-ci proposerait alors un service d'analyse des images rejetées et répétées lors d'un examen, dans le but de fournir aux hôpitaux un outil de suivi permettant d'améliorer l'efficacité des mammographies.

Dès lors, pour atteindre cet objectif le stage a consisté à :

- Concevoir un micro-service de gestion de données, grâce à une Interface de Programmation Applicative (API) et une base de données.
- Développer les interfaces logicielles et utilisateurs du mammographe en leurs permettant de communiquer avec le micro-service.
- Gérer l'échange des données en les postant et en les récupérant via une "API Rest".

Entreprise : General Electric Healthcare

Lieu : Buc, Yvelines, France

Responsable : M. Servant

Mots clés

- Recherche appliquée, développement
- Biens d'équipements professionnels
- Génie logiciel
- Analyse des données – Logiciels

Remerciements

Tout d'abord, je voudrais remercier M. Julien LEGER, manager de l'équipe Software Apps mammography, qui m'a permis d'effectuer ce stage au sein de son équipe. Je remercie également mon maître de stage, M. Nicolas SERVANT, Lead Engineer chez General Electric Healthcare, qui m'a inclus au sein de son équipe, m'a fait participer aux réunions et a su m'aider au quotidiens.

Je souhaiterais ensuite adresser mes remerciements au corps professoral et administratif de l'Université de Technologies de Troyes, pour son aide, ainsi qu'à M. SALEMBIER Pascal, enseignant référent sur ce stage.

Je remercie également M. SQUILLACCI Edouard, M. PEZET Anthony, M. DATIN Jean-Christophe et M. LECORGNE Bruno, ingénieurs architectes, pour leur accompagnement, leurs conseils et toutes les réponses à mes questions.

De plus, je suis énormément reconnaissant envers l'équipe d'intégration M. MILLEPIED Pascal et Mme PINOULT Sophie, pour avoir toujours été d'une aide très précieuse.

Je voudrais enfin exprimer ma gratitude envers mon équipe Agile M. DUVERLIE, Mme. BOURSIER, Mme MANIGOLD, M. MEHEUST et M. KANTAOUI. Ainsi qu'à tous mes collègues de l'équipe Software Apps Mammography, pour leur accueil, leur bonne humeur et leur bienveillance.

Sommaire

Remerciements.....	2
Sommaire	3
Présentation	4
Introduction.....	5
I. General Electric.....	5
II. GE Healthcare.....	6
III. GE Healthcare Buc, pôle d'excellence.....	7
IV. Service Software Apps Mammographie.....	7
Stage	8
I. Analyse des images rejetées et répétées.....	8
1) Mammographie.....	8
2) « Repeat Reject Analysis »	9
II. L'équipe "Software Apps"	13
1) Développement « Agile »	13
2) Ma place dans l'équipe.....	14
III. Déroulement du projet.....	15
1) Le développement du service RRA.....	15
2) Outils et méthodes	28
3) Résultats	34
Conclusion.....	46
Bibliographie.....	47
Table des illustrations	48
Annexe.....	49

Présentation

La mammographie numérique est de plus en plus utilisée de nos jours, afin de détecter au plus vite les cancers du sein. General Electric (GE) Healthcare développe des mammographes et leurs logiciels associés, qui représentent la modalité standard pour le dépistage du sein.

Dès lors, certains hôpitaux requièrent un suivi permettant d'améliorer l'efficacité des examens mammographies. Pour cela, GE Healthcare propose un service d'analyse des images rejetées et répétées lors d'un examen.

Le stage consiste donc à développer un micro service connecté au reste de l'application logicielle du mammographe. Celui-ci sera en charge de gérer efficacement une base de données. Il faudra par la suite travailler sur les interfaces logicielles et utilisateurs du mammographe leurs permettant ainsi de communiquer avec le micro-service afin de poster et récupérer les données.

Ce projet s'inscrit dans le cadre du stage de quatrième année du cursus Ingénieur en Informatique et Systèmes d'Informations de l'Université de Technologies de Troyes. Il se déroule au sein de l'équipe de Recherche & développement Software Apps Mammography de GE Healthcare, à Buc.

Introduction

I. General Electric

Son histoire

General Electric est une firme Américaine fondée en 1892 par Thomas Edison, l'inventeur de la lampe à incandescence. Au cours du XXème siècle, l'entreprise va croître et diversifier ses activités, devenant ainsi un leader de l'industrie mondiale. Elle sera moteur au développement de nombreux produits électroniques, allant des technologies médicales, du réfrigérateur, des réacteurs d'avions et nucléaires, à la radio et la télévision.

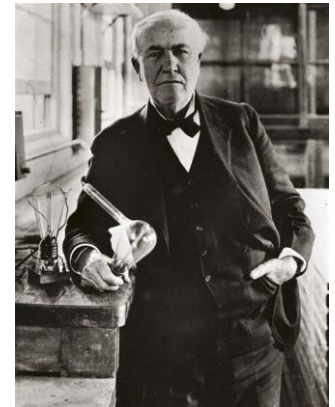


FIGURE 1 – THOMAS EDISON, 1920

Sa place à l'international

Aujourd'hui, General Electric regroupe des entreprises de différentes activités. L'entreprise comprend trente-six filiales situées dans plus de 150 pays, tandis que son siège social se situe, depuis juin 2016, à Boston (Massachusetts, Etats-Unis). Outre sa présence à l'international, la société représente en 2019 la 21ème fortune des entreprises Américaines (4ème en 2010), avec un chiffre d'affaires de 125 milliards de dollars et une place importante à la bourse. A titre indicatif, en 2016, Ge employait 295 000 personnes et se hissait à la huitième position dans le classement des vingt plus grosses capitalisations boursières. Son PDG actuel est Henry Lawrence Culp Jr, qui a succédé à Jeffery Immelt en 2017.

Depuis 2008, l'activité de General Electric se décompose en cinq branches principales :

- **GE Commercial Finance – GE Capital** : services financiers du groupe pour les particuliers et entreprises.
- **GE Healthcare** : branche médicale spécialisée dans le développement de solutions de haute technologie
- **GE Industrial** : produits industriels et services (éclairage, systèmes de sécurité, électroménagers ...)
- **GE Infrastructure** : réunion des branches GE Power, GE Oil & Gas et GE Energy Service
- **NBC Universal** : diffusion de chaînes de TV.



FIGURE 2 – GENERAL ELECTRIC EN FRANCE

Sa place en France

A l'image de l'acquisition récente du groupe français Alstom Energie et Réseaux, GE est un acteur économique majeur dans l'Hexagone. En France, GE compte 2 sièges mondiaux et 3 sièges européens. Ses principaux sites de productions sont situés à Belfort (GE Power and Water), à Buc (GE Healthcare) et au Creusot (GE Oil & Gas), faisant ainsi de ce pays le 5ème dans lequel GE est le plus implanté.

II. GE Healthcare

Actuellement, la filiale General Electric Healthcare a prévu une séparation de l'entreprise General Electric. Elle est réputée pour ses connaissances en imagerie médicale et son savoir en technologie médicale.

Sa création

- Lors de l'année 1920, General Electric acquérait la "Victor Electric Compagny", une société productrice de générateurs électrostatiques pour les tubes de rayons-X et les appareils électro-thérapeutiques à Chicago.
- L'utilisation des rayons X durant la Seconde Guerre Mondiale améliorera les capacités de production et d'expertise de la société menant GE Healthcare à la compagnie réputée pour ses connaissances en termes d'imagerie médicale et son savoir en technologie médicale qu'elle est aujourd'hui.
- Enfin, c'est en 2001 lors de l'acquisition d'Imatron, une société de Tomographe et d'imagerie Médicale, qu'elle prendra le nom General Electric Healthcare.

Situation à l'Internationale

Aujourd'hui, GE Healthcare est l'un des leaders mondiaux de la fabrication d'équipements d'imagerie médicale. Son siège social est situé à Chalfont St. Giles au Royaume-Uni. La compagnie emploie près de 50000 personnes dans plus de 100 pays, et propose des produits de traitement du cancer, des maladies cardiaques, des maladies neurologiques et d'autres pathologies. Ses concurrents principaux sont les groupes internationaux Siemens, Philips et Hologic, tandis que ses principaux sites dans le monde sont situés à Milwaukee (USA), Buc (France) et Tokyo (Japon), tandis que ses activités sont réparties selon 9 domaines :

- ❖ **DGS : Détection and Guidance Solution** : Appareil à Rayon X, Ostéodensitométrie, mammographe digitales (produit sur le site de Buc, France)
- ❖ **HD : Healthcare Digitale** : Produit les informations technologiques cliniques et financières requises pour chacun des produits, à l'image du GE Heath Cloud récemment développé par le groupe.
- ❖ **LS : Life Science** : Technologies pharmaceutiques, cellulaires et recherche pharmaceutique pour le diagnostic d'imagerie médicale.
- ❖ **LCS** : Life Care Solutions : spécialisé dans les appareils de soins intensifs tels que les électrocardiogrammes, les appareils d'anesthésie ou même les soins intensifs néonataux.
- ❖ **MICT** : Molecular Imaging & Computed Tomograph, développant les tomographes ainsi que la technologie d'imagerie moléculaire.
- ❖ **MR** : Magnetic Resonance, chargée de gérer les appareils à Imagerie par résonance magnétique (IRM).
- ❖ **US : UltraSound** : axé sur les produits d'imagerie à Ultrasons, utilisés par exemple en cardiologie.
- ❖ **Surgery** : Met à disposition des outils et des technologies assistant les interventions chirurgicales
- ❖ **Global Services**

Situation française

« GE Healthcare est l'un des leaders mondiaux de la fabrication d'équipements d'imagerie médicale. Présent en France depuis 1987, il emploie aujourd'hui 2600 collaborateurs, dont 400 ingénieurs R&D dans son site d'excellence internationale à Buc dans les Yvelines. GE Healthcare a noué de solides partenariats de recherche avec des PME et des centres de recherche français pour développer des technologies et des services médicaux révolutionnaires qui ouvrent une nouvelle ère pour les soins apportés aux patients. »

gehealthcare.fr

III. GE Healthcare Buc, pôle d'excellence

Pôle historique

Durant l'année 1987, alors que le groupe se nomme à l'époque GE Medical Europe, celui-ci fusionne avec CGR (équipementier Français du médical) et installe son Quartier Général à Buc, France.

Dès lors, le site sera utilisé comme "pôle d'excellence" en termes d'imagerie médicale, il contiendra son propre espace de production, des chercheurs, des ingénieurs, des équipes marketings mais également la direction générale. Il représentera ainsi plus de 2000 personnes sur ce site.

Le rôle du site

Au sein de GE Healthcare il existe différentes familles de technologies comme par exemple les ultrasons, les IRMs (Imagerie par Résonance Magnétique), les scanner ou la radiologie. A Buc, le service radiologie regroupe les différentes technologies utilisant les rayons X. On y trouvera ainsi les services responsables de la recherche et de la production des procédés vasculaire, radiologies, fluoroscopie et mammographie.

Les différentes parties d'un appareil de mammographie sont produites dans 5 pays différents. Cependant, la principale partie de la production du mammographe se situe à Buc. Au sein de la chaîne de production sont fabriqués les tubes à rayons X. Les équipes Hardware sont chargés de l'équipement matériel, mécanique électrique et électronique des machines. Enfin, les équipes Software ont pour but de développer les logiciels nécessaires au bon fonctionnement du mammographe (traitement d'images, interface utilisateur, gestion de la station de pilotage...). Pour finir, le site comprend la direction et les commerciaux.

IV. Service Software Apps Mammographie

Sa place dans l'entreprise

Le rôle du service software est de développer et d'améliorer les services informatiques proposés par les mammographes. En effet, ce sont les logiciels que les différents services mettent en place qui permettront aux produits de se démarquer. Les algorithmes de traitements d'image seront alors un moyen de dépasser les produits concurrents, mais également d'avancer dans le domaine médical de la mammographie. Pour finir, leur travail facilitera grandement celui des technologistes tout en améliorant les possibilités lors des examens en mammographie.

Détails du service

L'équipe Software Apps Mammography, dirigée par M. Julien LEGER compte environ 25 ingénieurs. Chargée du développement de la station d'acquisition, celle-ci représente la partie développement dans les recherches et le développement. Leur travail correspond à la couche applicative finale et au premier rendu visuel qu'auront les cliniciens gérant l'examen mammographique, la station d'acquisition du mammographe Pristina. Elle permet de choisir le mode d'acquisition d'images (routine ou biopsie), d'effectuer les réglages à distance, de prendre les images, les afficher sur l'écran puis de les stocker ou envoyer à une station de revue.

Les ingénieurs composant l'équipe sont tous des ingénieurs purement logiciel. Ces ingénieurs logiciel accompagnés d'Ingénieurs Intégrateur, d'architectes logiciel, et de manager de projets logiciels. La quasi-totalité du service Software Apps travaille dans un open-space afin de faciliter la communication.

Stage

I. Analyse des images rejetées et répétées

1) Mammographie

Contexte

En 2017, près de 60 000 nouveaux cas de cancer du sein ont été diagnostiqués, pour 12 000 décès causés par celui-ci la même année. En effet, le taux de mortalité causé par ce cancer a grandement diminué ces dernières années et pour causes, sa détection est nettement plus efficace. Actuellement, on compte plus de 87% des patients encore en vie 5 ans après un diagnostic.

La mammographie est un examen de détection de cancer du sein. Il utilise les rayons-X à basse énergie afin d'obtenir une image sur le même principe que celui d'une radio. En effet, les rayons ne traversent pas de la même manière les différents tissus, graisses, vaisseaux mammaires. Ces rayonnements sont ensuite captés et traités par ordinateur afin d'améliorer les images, puis, proposer celles-ci pour un diagnostic.

En France, un dépistage est fortement conseillé pour les femmes tous les 2 ans à partir de 50 ans. Cela permet de détecter l'anomalie au plus tôt et de mieux la traiter. Ce cancer touche particulièrement les femmes mais peut également impacter les hommes puisqu'environ 500 cas sont diagnostiqués chaque année, ce qui donne 0,5% du total des cancers masculins.

Procédure

Pour mieux comprendre le fonctionnement d'une mammographie, il est important de connaître la procédure de celle-ci. Le dépistage est réalisé par une technologue à l'aide d'un mammographe et d'une station d'acquisition d'image accompagnant celui-ci.

Durant l'examen, la technologue met en place la patiente et compresse le sein avec une "pelote de compression". Cela permet à la fois d'avoir une image de meilleure qualité grâce à l'étalement des tissus mammaires, mais également de diminuer la dose de rayons-X envoyés.

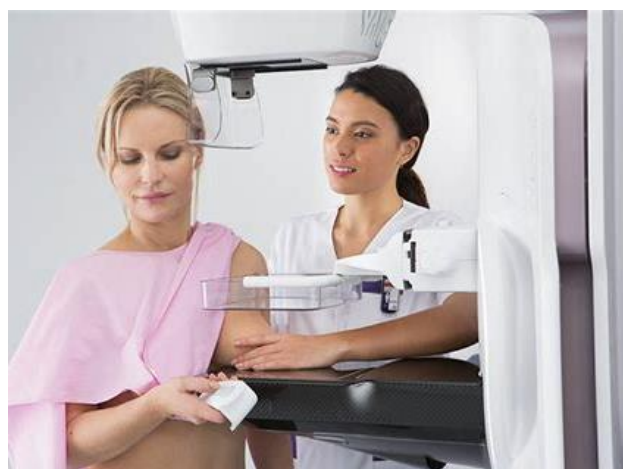


FIGURE 3 – MAMMOGRAPHIE AVEC PRISTINA

Par la suite, une faible dose de rayons-X est envoyée dans le sein et capturée par un détecteur. L'image peut ensuite être retravaillée et améliorée par des algorithmes de traitement d'image créés par une équipe "Image Quality". Les images obtenues (au minimum deux, une pour chaque sein) seront ensuite envoyées à un radiologue possédant des écrans plus adaptés. En cas de résultat négatif (suspicion de cancer), des examens plus poussés seront alors nécessaires : une biopsie peut alors être envisagée.

Mammographe Pristina

Pristina est le mammographe premium actuellement développé et commercialisé par General Electric Healthcare. C'est le mammographe premium qui se distingue de « CrsytalNova » un modèle "performance" moins coûteux destiné aux pays émergents. Prsitina se distingue des autres mammographes par plusieurs innovations :

▪ Design et ergonomie : le dispositif est plus agréable et moins anxiogène pour la patiente. Une molette actionnée par celle-ci permet une compression du sein plus importante que lors d'une compression effectuée par le radiologue, ce qui améliore en plus la qualité d'image. La patiente a donc une prise sur le déroulement de sa mammographie. De nombreux retours ont montré que les patientes trouvaient l'examen bien moins douloureux qu'avec d'autres dispositifs médicaux.



FIGURE 4 –PRISTINA ET SA STATION D'ACQUISITION

▪ Possibilité de réaliser des images 2D et 3D : Une partie articulée de la machine permet de prendre plusieurs images sous différents angles. Un logiciel de reconstruction rassemble les images afin de créer une vue en trois dimensions du sein. Cela permet notamment de placer précisément des points de repère afin d'effectuer une biopsie (prélèvements dans le sein).

▪ Plus faible exposition aux rayons X : La dose de rayons X délivrée par la machine est moins importante que pour d'autres dispositifs, et produit une image de haute qualité.

▪ Possibilité de réaliser des angio-mammographies : Il est possible d'injecter un produit de contraste à la patiente, afin de mieux déceler des anomalies, comme des tumeurs. Dans ce cas, deux images sont prises et recombinaison, afin de n'afficher que le produit de contraste, qui met en valeur le réseau de vaisseaux sanguins.

▪ Mode « routine » et « biopsie » : Le mammographe Pristina permet deux modes d'utilisation. Le premier, la « routine », correspond aux mammographies classiques de dépistage. Pendant cet examen, si le praticien détecte une anomalie, il peut proposer à la patiente d'effectuer une biopsie du sein (1 personne sur 10). Pour cela, il peut utiliser le mode « biopsie », qui permet d'effectuer des images en trois dimensions, de placer précisément des points de repère, et de prélever les échantillons voulus grâce à une aiguille placée sur la machine.

2) « Repeat Reject Analysis »

Les besoins : Bête à corne du sujet

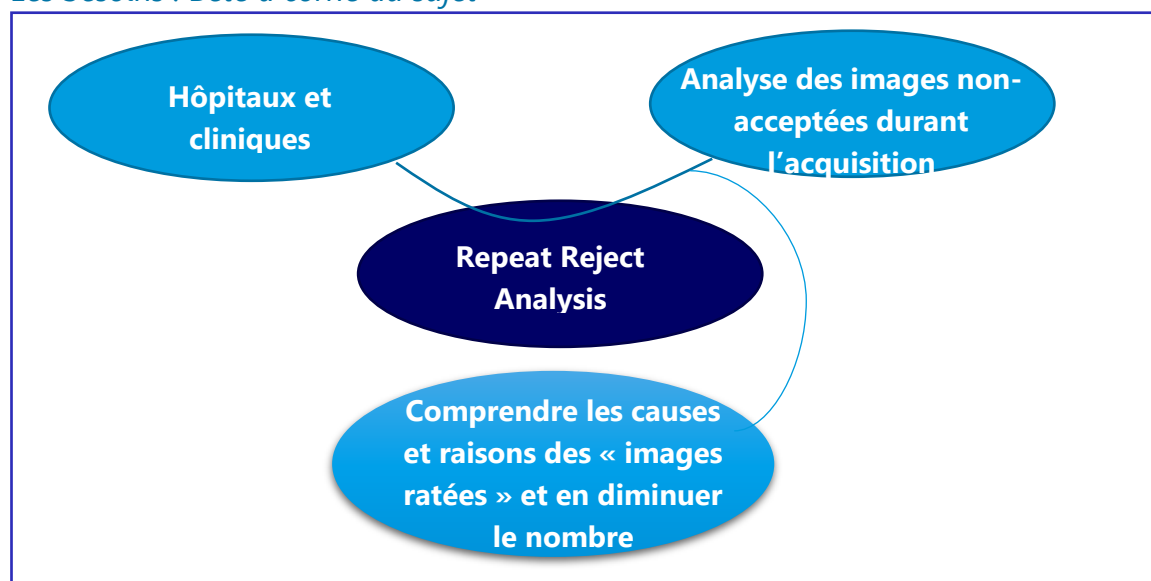


FIGURE 5 -BETE A CORNES DU PROJET

Utilisation

Tout d'abord, GE Healthcare est présent sur le marché International, ses mammographes peuvent ainsi être vendus à travers le monde. Il y a donc un nombre important de cliniques et hôpitaux ciblés qui peuvent chacun présenter un fonctionnement différent. Ainsi, il arrive parfois que ces établissements aient besoin de chiffrer le nombre d'acquisitions d'images réussies, et de même, d'images non utilisables pour la détection de cancer. Ces images non utilisables devront alors être reprises, ce qui coutera à l'établissement du temps et de l'argent. Le fait de chiffrer ces acquisitions dites « rejetées » ou « répétées » permettra d'analyser les causes du problème et d'y remédier par la suite.

C'est ce que permet le service d'analyse d'images répétées et rejetées, aussi appelé Repeat Reject Analysis ou encore RRA. Le fonctionnement de celui-ci est simple : lors de chaque examen de mammographie, la technologiste va faire un certain nombre d'acquisitions d'images. Chacune des images présentera un statut RRA (Accepté, Répété, Rejeté). A la fin de l'examen, un événement RRA par image acquise sera envoyé sur une base de données. Les événements pourront ensuite être récupérés par le logiciel, tous les mois par exemple, afin de fournir un bilan des examens.

Ce bilan sera créé de sorte à mettre en valeur les examens « ratés » et surtout la raison de ces échecs. En effet, si une acquisition est non utilisable, le statut RRA de l'image ne sera plus « Accepté » et la technologiste devra entrer les raisons du raté (nous détaillerons cette étape plus bas). Enfin, le fait de connaître les raisons de ces échecs lors des acquisitions permettra de fournir aux technologistes des formations plus spécifiques, plus cohérentes et donc plus efficaces.

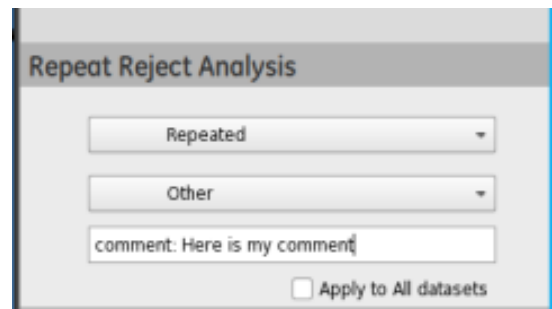


FIGURE 6 -VUE DE RRA SUR LA STATION D'ACQUISITION DU MAMMOGRAPHE

Détails

- Premièrement, un événement RRAEvent ou « Repeat Reject Analysis event » utilise un statut afin de connaître son état : Accepté, Rejeté, Répété.
 - Accepté, correspond à une acquisition utilisable pour la détection de cancer mammaire
 - Rejeté, signifie que l'image est inutilisable, si la patiente bouge pendant l'acquisition par exemple, l'image ne sera alors pas du tout traitée.
 - Répétée, précise que l'image est en partie utilisable, mais que pour une certaine raison, la technologiste a jugé bon de refaire une meilleure acquisition. Cela peut être le cas pour une image présentant une partie utilisable et une partie manquant de détails.
- Deuxièmement, un événement RRA a besoin d'être identifié. Il comprend ainsi un identifiant, l'identifiant de l'image à laquelle il est lié et la date de l'acquisition. Cela permettra par la suite de gérer simplement l'ensemble des objets RRA sur la base de données et de sortir des bilans par date. L'identifiant prendra la forme d'un Identifiant Unique et Universel appelé UUID (Universal Unique Identifier), un identifiant standard composé de groupes de caractères hexadécimaux en minuscules séparés par des tirets tel' que : « 180e8300-e28b-14e4-a736-4412459871505 ». L'image, elle sera identifiée grâce à son irradiationEventUID, un numéro confidentiel associé à chaque image 2D ou 3D.
- Troisièmement, un événement contient les données nécessaires à l'analyse des images non acceptées. Cela se traduit par différentes informations en plus du statut telles que la raison du rejet de l'image, un commentaire optionnel expliquant le rejet, ainsi que le contexte d'acquisition de l'image : Vue choisie,

procédure suivie, mode d'acquisition. Des listes prédéfinies seront alors définies afin de décrire les raisons et le mode d'acquisition. Toutes ces informations seront remplies par la technologiste au moment du rejet de l'image et pourront être utilisées la plupart du temps pour le bilan. Mettant ainsi en avant des difficultés à réaliser certaines vues, certaines procédures lors d'une acquisition.

- Pour finir, le nom du mammographe ainsi que celui du technologiste seront renseignés pour chaque événement, cela permettra un tri au moment de l'analyse, ou encore de sortir un bilan spécifique à une machine ou à un technologiste.

La plupart de ces données seront générées automatiquement par le logiciel qui les récupérera lui-même, cependant les informations sur les rejets seront ajoutées par le technologiste via une interface utilisateur. En parallèle, une nouvelle interface utilisateur permettra de communiquer avec la base de données et de générer différentes analyses des acquisitions non acceptées. Les personnes souhaitant générer une analyse sélectionneront alors une période à analyser, pourront préciser une machine et/ou un technologiste, tout en filtrant selon différents contextes d'acquisitions de l'image. Un rapport sera alors fourni et pourra être exporté au format PDF. Pour finir, le service "Repeat Reject Analysis" sera proposé sur chacun des mammographes mais sera optionnel et ne sera utilisé que dans certains pays ou certaines cliniques. A titre informatif, le plus gros utilisateur de ce service est actuellement les Etats-Unis.



Reason	Accepted	Repeated	Rejected	Non accepted percentage
BLANK	2	0	0	0%
CLINICAL_ARTIFACTS	0	0	0	0%
POOR_COMPRESSION	0	0	0	0%
WRONG_VIEWNAME	0	0	0	0%
Total	2	0	0	Non accepted: 0%

FIGURE 7 -EXEMPLE D'ANALYSE DES IMAGES REJETEES ET REPETEES (SUR 2 ECHANTILLONS ACCEPTES)

Sujet

Dès lors, le but de ce stage était le développement d'un micro-service service d'analyse des images rejetées et répétées sur les mammographes. Il existait une version précédente de ce service mais celle-ci n'était pas optimale, ce stage était l'occasion de refaire un service avec une architecture et un design neuf. Ce stage était ainsi un moyen de mettre en application les recherches récentes apportées en technologies digitales sur le modèle des micro-services par les architectes logiciels.

Parallèlement à la mise en application des micro-services, ce stage a permis d'utiliser de nouvelles technologies de tests, de base de données, d'intégration... En effet, le principe du sujet étant de mettre en place un service utilisable (dans la mesure du possible) à la fin du stage, il était important de définir à l'avance les technologies qui allaient être utilisées par la suite par l'équipe de développement (et non pas forcément les technologies actuellement utilisées).

Enfin, le sujet est resté le même tout au long du stage et est resté très cohérent par rapport à celui défini, cependant des modifications ont pu être apportées tant sur l'utilisation de ces technologies et de l'architecture, que sur les attentes quant au sujet. Ainsi, les données d'un événement RRA décrites ici ne sont pas forcément les mêmes qu'au commencement. De nouveaux besoins ayant été apporté durant le

stage, l'architecture a pu être modifiée pour y répondre, ou encore, de nouvelles attentes ont pu venir compléter les technologies utilisées.

De même, ces données décrites pourront, et seront probablement, légèrement modifiées d'ici la fin de mon stage. Actuellement, l'objectif premier est de finaliser le micro-service et de le rendre totalement fonctionnel. Cependant, si d'ici la fin du stage (début janvier 2020, tandis que ce rapport sera complété fin novembre 2019) les objectifs sont atteints, l'idée est de pouvoir étendre le service à un autre produit (« XRay »), certaines données seraient alors ajoutées ou modifiées afin de correspondre au produit ciblé. Il y aurait ainsi des changements de design mais l'idée serait de garder la même architecture, même structure et de l'appliquer aux besoins de ce produit.

- Pour conclure, le sujet de ce stage était donc le développement d'un micro-service d'analyse des images non-acceptés lors d'un examen de mammographie, ainsi que des composants communiquant avec ce micro-service. Celui-ci suivait une architecture et un design bien précis et défini par les architectes de l'équipe. De plus, il utilisait des technologies déjà mises en place dans l'équipe, mais également de nouvelles qui seront à l'avenir intégrés très probablement par l'équipe de développement. Ce stage a donc été l'occasion, en plus de concevoir un nouveau service, de mettre en application de nombreux outils pour le futur.

II. L'équipe "Software Apps"

1) Développement « Agile »

Durant toute la durée de mon stage, j'ai eu la chance d'intégrer une équipe de développement de Software Apps.

L'équipe

Software Apps est composée de trois équipes de développement, chaque équipe a la même structure et suit une méthode de développement Agile Scrum, cette méthode comprend trois rôles :

- Développeurs, dans ce cas-là des Ingénieurs Logiciel. Ils sont chargés de la gestion des logiciels, sa conception, son développement et ses tests
- Scrum Master, il est chargé de manager le groupe, d'en assurer la cohésion et de faire le lien entre l'équipe et l'extérieur (managers et autres équipes). Le scrum master dirigera les réunions d'équipe.
- Product owner, il aura pour rôle de s'assurer du bon respect des besoins du client. Il sera en contact avec l'équipe et transmettra les attentes des clients en veillant au respect de celles-ci.

Enfin, un architecte logiciel est relié à chaque équipe et sera chargé de fournir l'architecture et le design de chaque composant. L'ensemble des équipes est coordonné par un manager.

Son fonctionnement

Cette méthode permet la production en continu d'un logiciel, par multiples itérations. Le logiciel n'est pas conçu en une fois : il est découpé en un « backlog », qui regroupe toutes les fonctionnalités du logiciel. Celles-ci sont découpées en sous-fonctionnalités, implémentables rapidement. Leur définition constitue ce que l'on appelle « user story ». Celles-ci sont classées par priorité.

L'emploi du temps est découpé en « sprints », qui durent en général deux semaines. Chaque jour, le « stand up », une réunion d'une quinzaine de minutes avec l'équipe, permet de faire le point sur ce qui a été fait la veille, et ce qui est prévu de faire dans la journée. A la fin de chaque sprint, une nouvelle version du logiciel est livrée, et une réunion est prévue avec l'équipe afin de mettre en évidence les points positifs et négatifs, ainsi que des actions à effectuer pour s'améliorer au sprint suivant. Cela permet un développement efficace, et une bonne communication au sein du groupe.

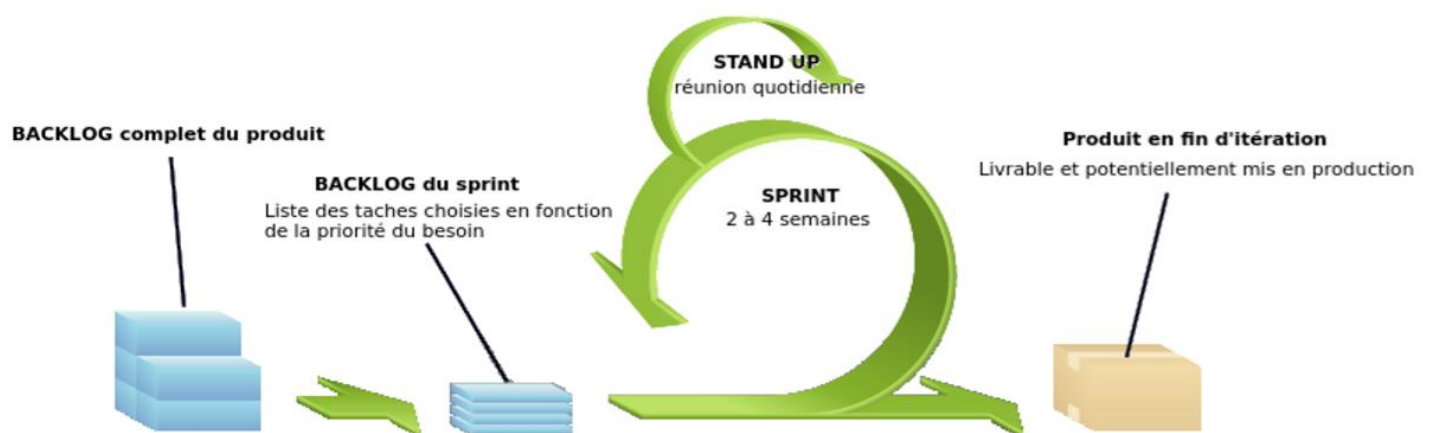


FIGURE 8 - ITERATION SELON LA METHODOLOGIE AGILE SCRUM

2) Ma place dans l'équipe

Rôle

Pour ma part, j'ai donc pu intégrer l'équipe de développement en tant qu'Ingénieur Software. Cependant, je n'avais pas les mêmes objectifs que les membres de l'équipe. En effet, l'équipe est amenée à travailler sur différents sujets sur un même sprint tandis que je me suis penché sur le même sujet, sujet de stage, durant mes 6 mois. Je ne pouvais donc pas non plus compter comme un membre à part entière de cette équipe puisque le sujet direct n'était pas le même et que mon apport ne présentait pas de valeur ajoutée à court terme.

Ainsi, je n'ai donc pas travaillé directement dans le groupe car mon projet était totalement à part, car planifié pour le futur. J'ai donc collaboré avec l'équipe non pas sous la forme d'un partenariat sur un projet commun mais plus comme de l'assistance entre collègues.

Apport

Ainsi, à l'inverse de l'équipe avec laquelle je travaillais, je ne rendais pas de travail (d'apport logiciel) à la fin de chaque sprint. Mon sujet de stage a été défini de sorte à prendre de l'avance sur les projets de la société. Ainsi, le thème que j'aborde actuellement, ne rentrera dans le plan de GE Healthcare que pour le développement du prochain modèle de mammographe, soit pour fin 2020. Ce stage est donc l'occasion pour l'équipe de prendre de l'avance.

J'ai également pu à travers ce stage, proposer un apport à plus court terme en mettant en application les nouvelles technologies sélectionnées par les architectes, tout en en développant pour le long terme une partie du logiciel futur, dans ce cas-là un micro-service et ses composants.

III. Déroulement du projet

1) Le développement du service RRA

A. *Situation de faits*

Besoin

Afin de replacer le contexte, voici un rapide résumé du projet. L'objectif était de développer un micro-service d'analyse des images rejetées et répétées. Celui-ci permet l'analyse des images non acceptées lors des mammographies permettant ainsi d'en comprendre les raisons et d'y remédier afin de diminuer au mieux les ratés. En effet, chaque examen coûte du temps et de l'argent aux cliniques et assurer des meilleures formations grâce à ces analyses permettrait de réduire ces coûts. De plus, un examen de mammographie est souvent désagréable pour la patiente à cause de la compression, le fait de devoir refaire une acquisition rajoute donc du temps de compression, et peut augmenter la douleur ressentie.

De plus, ce projet serait également l'occasion d'appliquer de nouvelles méthodes de développement pour le service afin que celui-ci puisse mettre en place ces nouveaux moyens par la suite. Le problème à résoudre serait ainsi de concilier les solutions techniques, proposées par les recherches digitales précédemment réalisées, avec les besoins du service d'analyse des images répétées et rejetées.

Pour finir, ce service est nécessaire pour le produit final, car même s'il est optionnel, il a été demandé par le client, les cliniques ou les hôpitaux, afin que celui-ci puisse suivre l'utilisation de son ou ses mammographes, dresser facilement un bilan des points à améliorer et ainsi conseiller au mieux les technologistes pour que le dépistage soit rapide et efficace.

Antécédents

De même, il est important de noter qu'une version de ce service était existante à mon arrivée. Cependant, ce service n'était pas optimal car il avait dû être réalisé rapidement. L'architecture de celui-ci était améliorable, et le rendu visuel ou le design de son interface utilisateur également. Enfin, le code réalisé est assez complexe et une modification de celui-ci, pour l'ajout d'une information par exemple, aurait pu s'avérer trop complexe.

Ainsi, l'entreprise souhaitait reprendre l'architecture du service tout en améliorant le design de celui-ci. L'objectif était de permettre au nouveau service d'être facilement modifiable selon les besoins, de pouvoir s'appliquer simplement sur différents mammographes via une amélioration de la portabilité, et en plus de nouvelles technologies, de présenter une meilleure interface utilisateur du rapport d'analyse, plus améliorée et visuelle, tout en améliorant la portabilité de ce rapport.

Les faits

Dès lors, j'avais à ma disposition pour modèle la version précédente du système d'analyse ainsi que le travail des architectes définissant le modèle à suivre pour le développement, les informations contenues par un évènement RRA et les technologies à utiliser (base de données, outils de tests...). De nombreux schémas explicatifs, de design ou encore du texte disponible sur des pages communes à l'entreprise étaient ainsi disponibles au commencement de mon stage. De même, l'ingénieur architecte digital avait auparavant réalisé un gros travail pour la mise en place de micro-service, que j'ai pu utiliser pour développer mon service.

Par ailleurs, il a fallu modifier une partie du logiciel du mammographe, celui-ci était bien évidemment déjà existant et présentait une documentation importante. Le logiciel était codé en C++, et utilisait une interface Qt. Afin de pouvoir tester le code produit, nous avons à disposition une Machine Virtuelle (tournant sous linux), utilisée pour lancer une simulation de mammographie. Celle-ci permettait de tester sur des conditions presque réelles, et d'avoir un rendu visuel efficace.

Pour finir, l'objectif du stage pour produire ce service RRA était :

- D'abord de créer un Micro-Service de gestion de donnée, appelé RRA Server, permettant le stockage de chacun des événements RRA.
- Puis de créer un second micro-service, RRA Gateway permettant la communication entre le RRA Server et le mammographe.
- Enfin, il fallait modifier le code du mammographe afin d'y ajouter de quoi saisir de nouveaux événements RRA à chaque examen.

Pour finir, il fallait créer une Interface Utilisateur dans le but d'obtenir l'analyse des images rejetées et répétées.

B. Solution proposée

Solution

Pour répondre aux objectifs, les architectes avaient mis à disposition au début du stage les diagrammes nécessaires pour comprendre les besoins et les réaliser proprement. On peut ainsi diviser le stage en 4 objectifs principaux.

- Premièrement, la réalisation du serveur collectant les données d'analyse.
- Deuxièmement, le développement de la Gateway, facilitant le lien entre ce serveur et le mammographe.
- Troisièmement, la modification du logiciel présent sur le mammographe pour y ajouter ma partie.
- Dernièrement, la conception de l'interface utilisateur permettant l'analyse des données collectées.

a) Le serveur

Tout d'abord, pour répondre à la problématique d'avoir une bonne interface pour ce stage, les architectes ont proposé l'utilisation d'une interface de programmation applicative, aussi appelée API. Celle-ci permet de faciliter la communication entre des applications ou des logiciels, afin de simplifier l'échange de données sans avoir à gérer le lien entre ces deux. Ainsi, l'API mettra en lien la Gateway, le serveur micro-service et l'interface Utilisateur en répondant à des standards dans chacune des communications.

L'Interface utilisée sera une API RESTfull, c'est-à-dire que les différents composants communiqueront entre eux grâce à des requêtes https. Ainsi, le micro service recevra des requêtes https de type 'GET' permettant de recevoir des données du micro-service, ou 'POST' permettant d'envoyer une donnée au micro-service.

Exemple de requête connue : « <https://zimbra.utt.fr/mail#7> »

Le serveur RRA est constitué d'un micro-service chargé des interactions permettant la gestion d'une base de données. Dès lors, son but est simple, recevoir les requêtes http, puis les traiter afin d'émettre ou de recevoir une ressource RRA.

La base de données permet la gestion d'un grand nombre de ressources. Elle peut stocker, recevoir, et renvoyer une ou plusieurs de ces ressources triées.

La ressource « **RepeatRejectAnalysis** » (Diagramme d'objet disponible dans l'annexe)

Chaque acquisition d'image par une technologiste aura son RRAEvent stocké dans la base de données.

Un objet RRAEvent est composé de la sorte :

- id : identifie de manière unique chaque objet, c'est un UUID.
- irradiationEventUID : Utilisé par le mammographe pour différencier chaque image ou ensemble d'image.
- dateTime : la date et l'heure au moment de la création de l'objet (diffère de celle de l'image).
- reason : indique la raison de la non-acceptance
- status : donne le statut de l'objet (accepté, rejeté ou répétée), sélectionné par la technologiste.
- OptionalComment : La technologiste peut ajouter un commentaire (optionnel) donnant plus de détails.
- clinicalView :
- acquisitionMode : indique si l'acquisition a été réalisée avec des paramètres automatiques ou manuels.
- imageType : Renseigne le type de procédure lors de l'acquisition de l'image
- protocolName : indique le nom du protocole au moment de l'acquisition
- technologistName : contient le nom du technologiste chargé de l'acquisition
- device : définit le nom de l'appareil utilisé pour l'acquisition.

Ces objets pourront être regroupés par pertinence, voici la représentation d'un objet RRAEvent au format JSON :

```
{ "RRAEvent": {  
  "image": { "irradiationEventUID": "1.2.840.10008.1.1.2.1.0.4.3.6" },  
  "properties": { "dateTime": "2019-11-05T09:16:22.554+0200" },  
  "data": { "status": "RRA_STATUS_REPEATED",  
    "reason": "RRA_REASON_POSITION" },  
  "comment": { "text": "Leave any optional comment here" },  
  "acquisitionContext" : { "acquisitionMode" : "MANUAL",  
    "clinicalView": "RCC",  
    "protocolName": " STEREO",  
    "imageType": " DERIVED\\PRIMARY\\STEREO_SCOUT" },  
  "device" : { "stationName": " Pristina" },  
  "technologist" : { "technologistName": "Dupont" }  
}
```

Le modèle décrit ci-dessus sera utilisé comme standard pour communiquer avec le micro-service, cela permettra une unification des procédés. Ainsi, le micro-service pourra recevoir une requête 'POST' contenant dans le corps de cette requête un RRAEvent, et une requête 'GET' vers le micro-service recevra un ou plusieurs RRAEvents.

Finalement, les différents services constituant RRA communiqueront entre eux en utilisant le modèle décrit et définissant l'API. Les messages envoyés et reçus seront donc toujours les mêmes et chaque composant saura comment les gérer sans avoir à s'occuper du fonctionnement d'un autre composant. Ce système permet une grande flexibilité. En effet, il permet d'utiliser n'importe quel type de composant, dans n'importe quel langage, et de pouvoir changer ce composant à tout moment du moment que l'API utilisée et donc les messages traités restent les mêmes. Pour finir, cette interface était parfaite pour l'utilisation de micro-service.

b) La Gateway

Afin que le mammographe puisse communiquer avec le micro-service RRA, il faut lui envoyer des messages respectant les standards définis dans L'API. C'est le rôle de la Gateway : récupérer les événements envoyés par le mammographe (via DDS, voir la partie suivante) et envoyer une requête POST au micro service pour créer ce nouvel RRA Event.

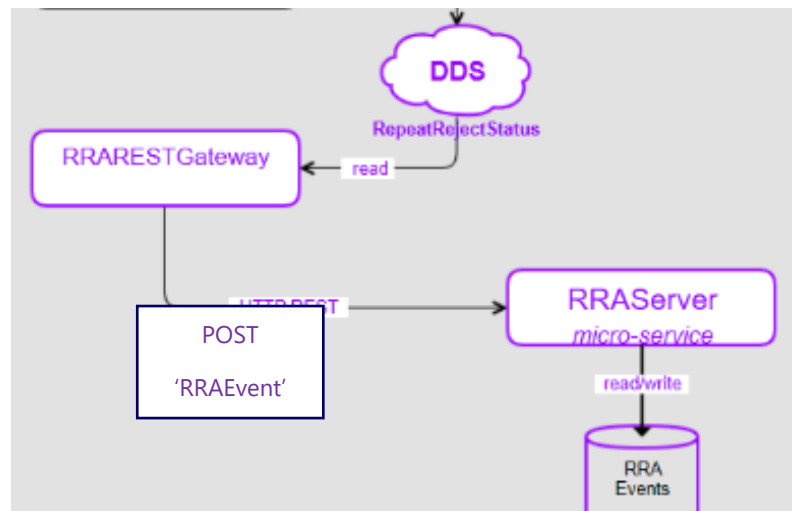
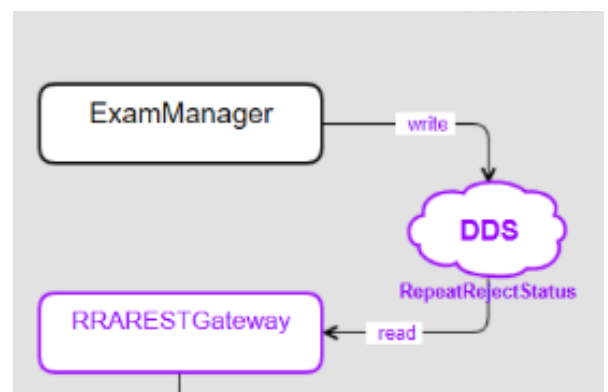


FIGURE 9 ET 10 : SCHEMAS DES COMPOSANT SERVEUR-GATEWAY-MAMMOGRAPHE

c) Le mammographe

La partie du Mammographe à modifier se nomme ExamManager, c'est ce composant logiciel qui gèrera les détails des acquisitions lors de l'examen. A la fermeture d'un Examen, ExamManager va créer un évènement RRA, et l'envoyer à DDS. Le 'bus' DDS est utilisé par ExamManager afin de transmettre des messages sans avoir à se soucier de qui l'envoie ou qui le reçoit, ce qui facilite le processus de communication. Ainsi, ExamManager va envoyer un messenger à DDS contenant le RRA. Il faudra donc en modifier une partie afin de récupérer les informations pour RRA déjà accessibles dans le code.



De plus, il faudra ajouter à l'interface du mammographe. L'espace contenant les boutons nécessaires à créer les nouveaux événements RRA, cette interface sera gérée dans l'AcquisitionViewer que nous détaillerons plus bas.

d) Interface utilisateur d'analyse

L'interface d'analyse de RRA, va être présentée sous la forme d'une page HTML5, ce qui permettra de l'exporter à différents endroits. Pour le moment, la page HTML sera située au niveau de l'IQTool, mais cette situation est amenée à être modifiée.

Cette page permettra de faire une analyse selon certains critères, d'afficher cette analyse, puis de l'exporter au format PDF. Afin de procéder à une analyse l'interface sera en contact avec le micro-service via des requêtes https 'GET'. La requête contiendra deux dates :

"startDate" et "endDate" qui

définiront la période sur laquelle on

souhaite faire l'analyse. La requête pourra également contenir de manière optionnelle un "deviceName" et un "technologistName" permettant de filtrer la recherche par appareil ou par technologiste.

L'interface permettra de créer cette requête en remplissant les champs d'informations selon les besoins.

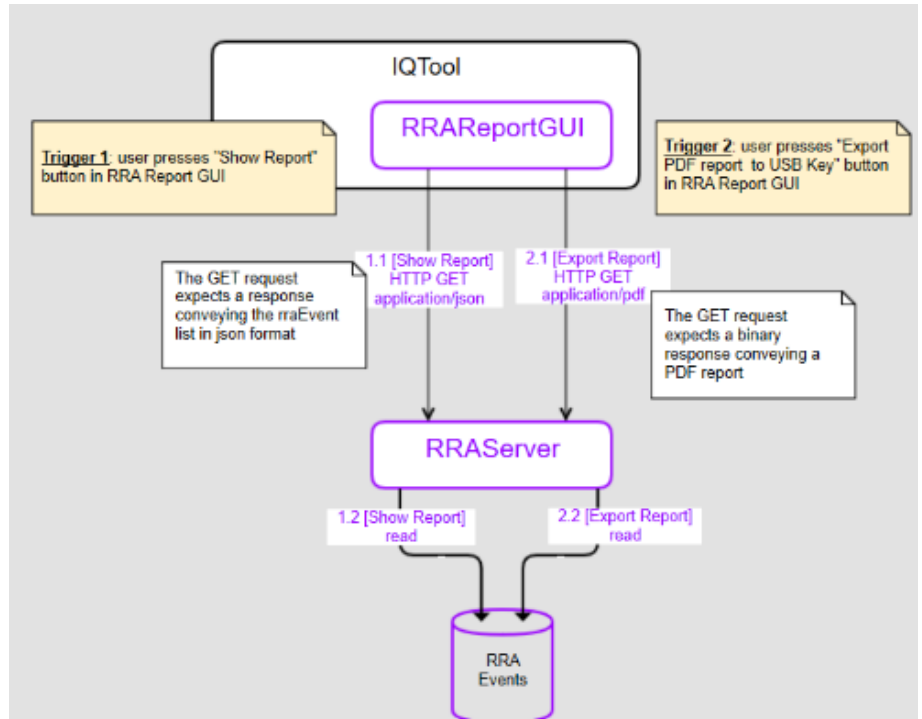


FIGURE 11 : SCHEMAS DU COMPOSANT DE L'INTERFACE UTILISATEUR D'ANALYSE

FIGURE 12 : PAGE HTML REPEAT REJECT ANALYSIS (FILTRE DE L'ANALYSE)

Exemple de requête http du micro-service :

<http://localhost:9080/v1/repeat-reject-analysis-events?fromStartDate=2019-10-28T09%3A16%3A22.554%2B0200&toEndDate=2019-12-28T09%3A16%3A22.554%2B0200>

Cette requête une fois le micro-service démarré renverra une liste de RRAEvents. Elle contient en arguments deux dates (les %3A... remplacent les « : » non autorisé dans des url et B0200 correspond au décalage horaire pour le microservice).

La requête permettra une réponse au format JSON, qui sera traitée afin d'obtenir une réponse visuelle grâce à des graphiques et des tableaux. Ces éléments sont disponibles en annexe.

Architecture

a) Le serveur

Le serveur est un micro service suivant une structure spécifique à ce genre de composant. L'idée est d'appliquer cette structure à tous les micro-services futurs afin de mettre en place un standard pour ce service. La structure est définie de la manière suivante : elle comprend une interface décrivant la ressource utilisée (ici un RRAEvent), tel un modèle, puis une application, qui comprend le code produit dirigeant le micro-service et gérant la ressource de l'interface. Enfin, l'acceptance est utilisée pour tester le bon fonctionnement du micro-service. Pour finir, les scripts sont présents dans le micro-service et sont utilisés tout au long du développement de celui-ci (de la génération de code jusqu'au démarrage).

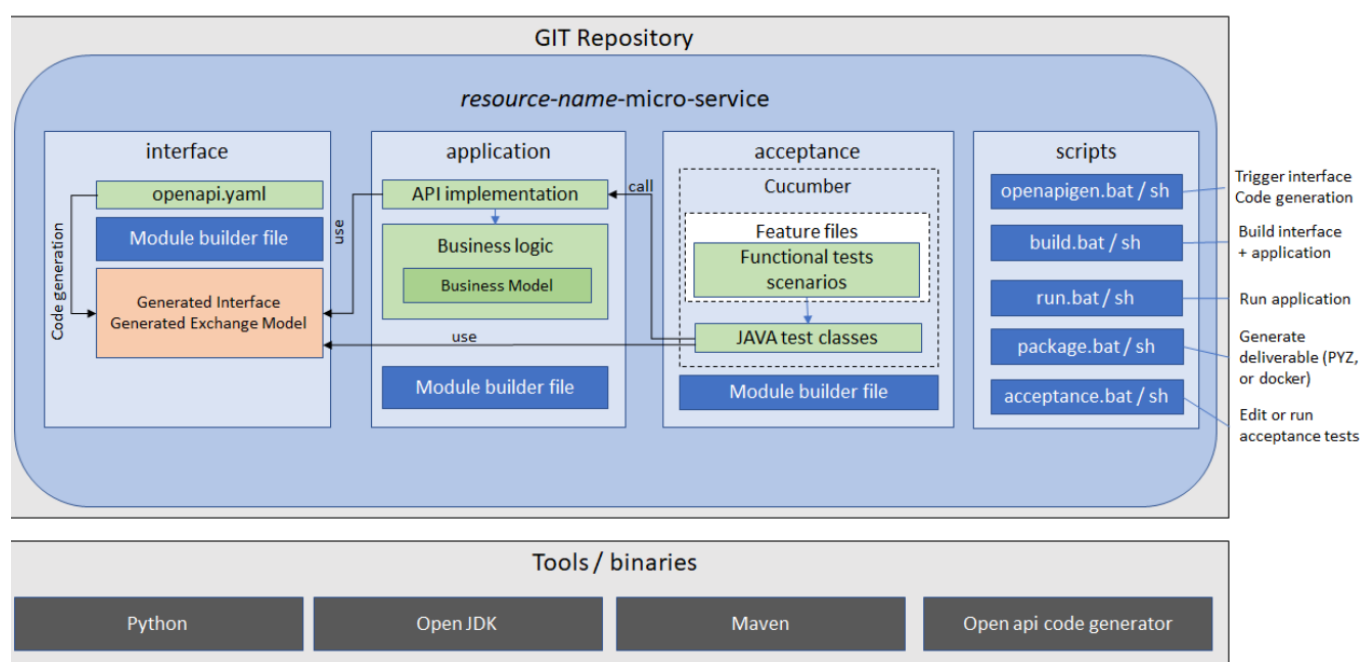


FIGURE 13 : STRUCTURE GENERAL D'UN MICRO-SERVICE DANS GIT

Nous allons ainsi décrire ces trois différentes parties de ce composant, les scripts étant plus détaillés dans la partie suivante.

Interface

Elle est chargée de décrire la ressource utilisée, ainsi que le fonctionnement de l'API. L'interface est composée de trois outils :

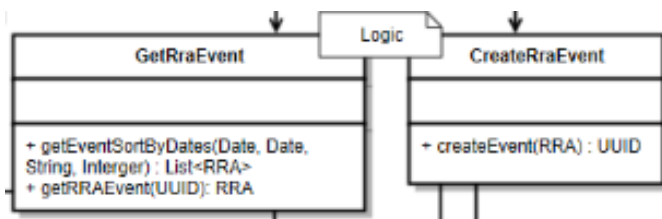
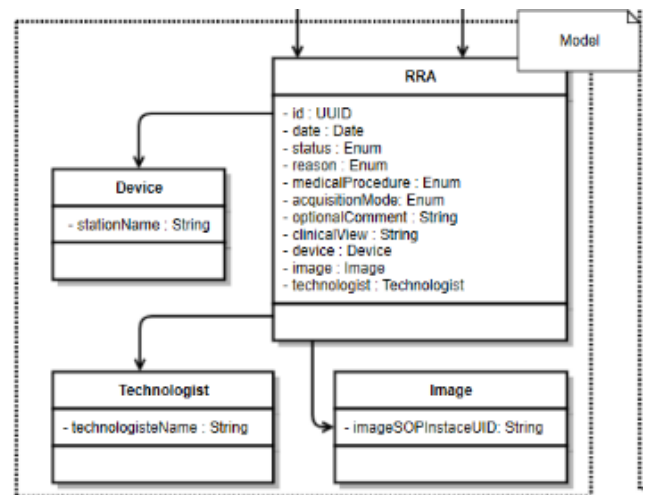
- Le fichier openapi.yaml, un fichier décrivant l'interface API REST du micro-service. Celui-ci va décrire la ressource RRAEvent du service ainsi que les requêtes http gérées par le micro-service.

- A partir du modèle fourni par le YAML, le code de l'interface du micro-service sera auto-généré par un script.
- Cela produira ainsi le modèle d'échange (la ressource) et le modèle de l'API (modèle d'Interface). Ce modèle comprendra chacune des méthodes de réponses aux requêtes http. Il y aura donc trois fonctions retournant chacune une réponse (type java : Response). Le code du modèle est auto-généré, il ne sera ainsi pas modifié. Pour modifier l'interface, on changera le fichier YAML et on relancera le script d'auto génération.

Application

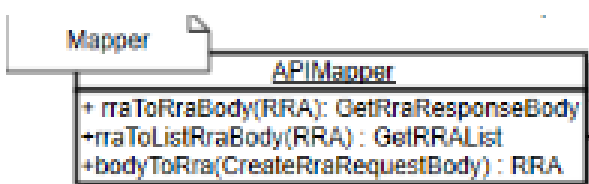
Une fois le modèle d'interface créé, on implémentera dans le fichier d'application le modèle de l'API (modèle d'interface). Cette implémentation appellera par la suite les méthodes de la logique. Voici les différents composants de la logique :

- « business model » : décrit la ressource au format de l'application.
Il est principalement composé de la classe "RRA". Celle-ci représente dans ses attributs toutes les données de la ressource et contient elle-même les classes "Image" comprenant un identifiant permettant de relier l'objet à l'image acceptée ou non, un "Device" ayant comme attribut le nom de l'appareil utilisé au moment de l'Acquisition et "Technologist" possédant le nom du technologiste chargé de l'acquisition.



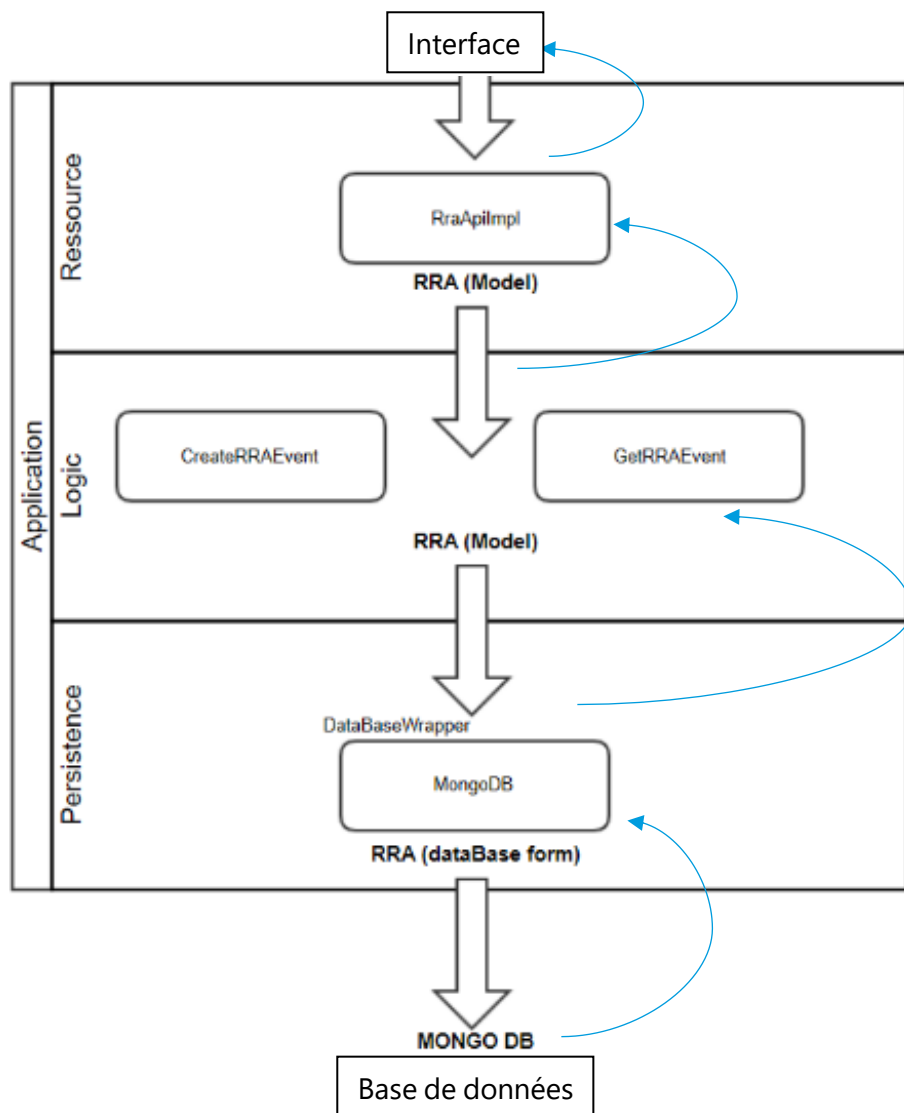
- « business logic » : contient les méthodes appelées pour l'implémentation de l'interface. Celles-ci sont partagées selon leur but dans les classes Get/Create(RRAEvent), ces méthodes vont à leur tour appeler les fonctions de la persistance afin de communiquer avec la base de données.

- La persistance : fait le lien entre la base de données et l'application. Elle utilise l'outil JPA, permettant de communiquer avec de nombreuses bases de données. Les méthodes appelées par la logique présenteront alors selon les besoins des requêtes au format SQL. JPA traduira ces requêtes pour les utiliser sur la base de données souhaitées.
- Le Mapper : Traduit la ressource RRA d'un format à un autre.



Ces méthodes prennent en argument un objet RRA du modèle de l'interface et le traduisent en objet RRA du modèle de l'application ou inversement. Il y a trois méthodes car le modèle de l'interface présente deux représentations différentes de l'objet selon la requête http ("ListRRA" étant un objet RRA plus léger et moins précis et inversement pour "RRABody").

Enfin, voici un exemple du "chemin" parcouru par la ressource d'un RRAEvent. Celui-ci est envoyé au micro-service via une requête http, celle-ci est récupérée par l'interface dans un premier temps, puis dans un second, par l'application et plus exactement par la partie Ressource implémentant l'interface. L'implémentation appelle alors les méthodes de la logique en lui fournissant la ressource RRAEvent au format du modèle. Ce format est créé par le Mapper. Puis, les méthodes de la logique appellent à leur tour la Persistance utilisant JPA, celui-ci va traduire la requête et la ressource de manière à communiquer avec la base de données. Chaque méthode retournera par la suite la réponse obtenue dans la base de données, allant ainsi de la base de données (MongoDB) jusqu'à l'interface via le chemin inverse.



Pour résumer : l'Interface comprend la description de l'API. L'Application va implémenter cette Interface comprenant les réponses aux requêtes https du micro-service. Pour cela, il appellera la business logique, qui utilisera à son tour les méthodes de la persistance pour communiquer (poster ou récupérer) avec la base de données. La ressource RRA utilisée sera décrite dans le business Model et la traduction depuis le modèle de l'interface se fera grâce au mapper.

Acceptance

Pour finir et dans le but de confirmer le bon fonctionnement du micro-service, le composant "d'Acceptance" du micro-service sera chargé de définir et d'exécuter de manière automatique des tests de vérification. Pour cela, les architectes ont choisi d'utiliser l'outil Cucumber (défini plus bas). Ce composant comprend ainsi un fichier de "Feature" chargé de décrire les tests à réaliser dans le langage cucumber, celui-ci est très proche du langage courant contrairement aux autres langages. Chaque test sera décrit dans le fichier de "Feature" par des scénarios et ces scénarios devront par la suite être implémentés par les classes de tests de JAVA.

Ainsi, le composant d'Acceptance comprendra un fichier Cucumber de scénarios décrivant les tests à réaliser, des classes java implémentant ces scénarios et un script permettant d'exécuter automatiquement les tests sur le micro-service.

Pour conclure sur le micro service, il comprend une interface autogénérée définissant le modèle à utiliser, l'application implémente ce modèle afin de remplir les besoins du micro-service face aux requêtes http, puis, l'acceptance teste le bon fonctionnement du service. Tout au long du développement, le micro-service sera géré sur un "Git Repository", une fois terminé, on changera simplement celui-ci de branche, marquant ainsi la fin d'une première version du micro-service.

b) La Gateway

Lorsque le micro-service est terminé, on va se pencher sur le rôle de la Gateway. Celle-ci va recevoir et traduire l'objet RRAEvent reçu du mammographe pour l'envoyer au format d'une requête 'post' http au micro-service pour que celui-ci le stocke. La Gateway est semblable au micro-service à l'exception que celle-ci ne gère pas de ressource mais s'occupe seulement de la transition de celle-ci. Son architecture sera cependant assez proche du micro-service du Serveur RRA. Elle comprendra ainsi : une partie Applicative, une partie d'Acceptance et un fichier de script. De plus, pour définir la ressource qu'elle traduira, la Gateway utilisera un fichier IDL qui autogénérera de quoi traiter la ressource RRAEvent.

IDL

Un IDL est un fichier utilisé pour décrire une ressource. Ici la ressource sera un RRAEvent. Chaque IDL sera un fichier comprenant une structure et les informations définissant chaque donnée selon un type. Les IDL peuvent faire appel à d'autres IDL à l'image des classes dans d'autres langages de programmation. Voici donc la description du fichier RepeatRejectAnalysisi.idl :

Il contient trois énumérations, soit trois listes indiquant les raisons, statuts et modes d'acquisitions possibles. Il est ensuite composé de la structure RRAEvent, contenant toutes les données d'un RRAEvent : les trois énumérations décrites précédemment, les attributs au format String (celui-ci a été redéfini par l'équipe de développement afin d'y rajouter d'autres paramètres tels que le nombre de caractères maximum). Enfin il contient également une clef servant à identifier l'RRAEvent. Le tout est regroupé dans un module appelé "dds".

```
module dds {  
    enum AcquisitionMode{  
        MANUAL, AOP    };  
    enum Reason{  
        patient_motion, poor compression...    };  
    enum Status {  
        Accepted, Rejected, Repeted    };  
    Struct RRAEvent{  
        AcquisitionMode acquisitionMode ;  
        SwAppsDD ::String irradiationEventUID ; //@key  
        ..... } } ;
```

L'IDL servira donc de modèle de ressource pour l'échange de données dans la Gateway.

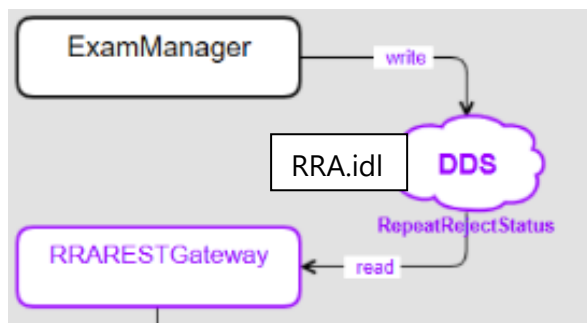


FIGURE 14 : RÔLE DES IDL ET DE DDS

DDS

Dès lors, le mammographe enverra un RRAEvent à la Gateway au format de l'idl via DDS. Le principe de DDS est de créer différents sujets sur lesquels peuvent s'inscrire des composants en tant que "lecteur" (reader) ou "notifieur" (writer). Sur chaque sujet, les notifieurs peuvent poster des messages au format IDL. Les lecteurs alors inscrits sur ce sujet recevront alors le message DDS toujours au format de l'idl. Ici, le mammographe est représenté par la classe "ExamManager". Celui-ci est un "Writer DDS" qui postera

un RRAEvent au format RRA.idl, DDS notifiera alors la Gateway inscrite comme "Reader DDS" qui lira l'RRAEvent au format RRA.Idl.

Application

Lorsque le message DDS est reçu par la Gateway, celui-ci va pouvoir être traité par la Gateway afin de traduire ce message de manière à l'envoyer au micro-service. La partie Appllicative de la Gateway comprend deux ensembles de classes qui seront appelés par la classe principale.

DDS.Client

Cette partie est chargée du lien DDS/Gateway. Lorsque la Gateway est notifiée par DDS, ce sont ces classes qui récupèrent et traduisent le message reçu. Elles peuvent donc lire la notification et créer un nouvel objet RRAEvent en fonction avant de l'envoyer au micro-service.

API.Client

Ces classes auront pour but d'établir la connexion Gateway/micro-service. Elles contiennent des fonctions JAVA capables de gérer les requêtes https. De plus, elles récupèrent les variables d'environnement de la Gateway indiquant les paramètres de la requête (l'hôte, le port, donnant par exemple <http://localhost:9080>).

Finalement, la Gateway étant proche du micro-service à l'image du serveur, celle-ci contient également des fichiers d'acceptance utilisant Cucumber pour valider le bon fonctionnement du composant, mais également des scripts, permettant de générer le code de la ressource (via l'IDL ici), de gérer le développement et de démarrer le composant. Pour finir, ce composant est également déposé régulièrement sur un dépôt Git pour la sauvegarde, une fois terminé, il est alors changé de branche marquant ainsi la fin d'une version.

c) Le mammographe

Le mammographe est la partie existante déjà utilisée par le service. Le composant se nomme SwApps car il est géré par l'équipe comme son nom l'indique et est assez complexe puisqu'il contrôle la station d'acquisition. Le code est en C++ et est sans cesse amélioré. J'aurai ainsi à modifier deux composants pour l'ajout du service d'analyse d'images rejetées et répétées :

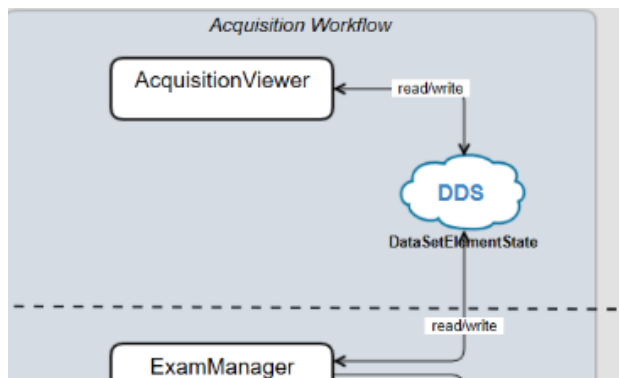


FIGURE 15 : SCHEMAS DES COMPOSANT UTILISES DANS LE MAMMOGRAPHE

AcquisitionViewer

Est la partie visuelle de la station d'acquisition. Elle comprend une partie en QML pour le design visuel pur, qui vient être complétée par une partie en C++ qui respecte un design pattern Model Vue Contrôleur. Celui-ci est utilisé afin de faire le lien entre ce que les technologistes voient ou font sur la station d'acquisition et les réponses que le mammographe envoie en fonction.

Acquisition Viewer et ExamManager effectueront un échange de données à l'aide de DDS. Ainsi, les premières informations nécessaires et saisies par les technologistes sur le mammographe seront envoyées à ExamManager via des "DataSetElementState" qui contiendra des informations sur une acquisition d'image.

ExamManager

Ce composant se chargera ensuite d'envoyer les informations relatives à une acquisition. Ainsi, c'est au travers de celui-ci que le mammographe enverra un RRAEvent à la Gateway. Il contient les classes nécessaires à créer de nouveaux objets, les enregistrer et des Readers/Writers DDS. ExamManager recevra les informations saisies et viendra y ajouter les informations disponibles avant l'acquisition et donc que celui-ci possédait déjà. Il créera ainsi l'objet original RRAEvent qu'il enverra à la Gateway via un dds Writer.

Au moment de la fin de mon rapport je termine tout juste ce composant.

d) Interface utilisateur d'analyse

L'interface utilisateur est faite pour être exportable, elle est ainsi en HTML5 pour son accessibilité. L'UI est donc composée du code HTML, décrivant la page ; d'un script JS chargé des interactions sur cette page et d'un fichier CSS améliorant le design qui sera complété par un fichier d'image contenant par exemple le logo de GE.

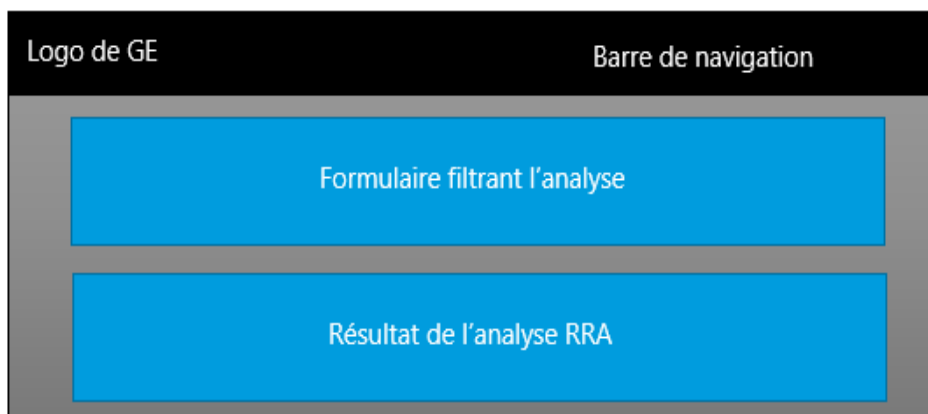


FIGURE 16 : SCHEMAS SIMPLIFIES DE LA PAGE HTML

- **HTML**

La page d'analyse des images rejetées et répétées se présente sous la forme suivante (image de la page disponible en annexe) :

- Une barre de navigation comprenant le logo et le titre, auxquels pourront s'ajouter des liens de navigation.

- Un premier panel contenant les champs à filtrer pour l'analyse.

-Et un second panel contenant les résultats de l'analyse, celui-ci apparaît seulement une fois la recherche lancée. L'analyse contient un tableau accompagné de diagrammes révélant les causes principales de rejets.

Ainsi, le design HTML affiche la barre de navigation, le formulaire et les résultats (c'est le script qui se chargera de rendre visible ou non l'analyse).

Filtrer les événements par :

Période Début Fin	Noms Appareil Technologiste	Groupe [Groupes à afficher] Filtrer Reset
--	--	---

Le formulaire est partagé en trois champs de filtre par :

- Période : Avec des champs de début et de fin, ces champs s'ajouteront comme argument dans la requête url) qui sera envoyé.
(Exemple : <http://localhost:9080/v1/repeat-reject-analysis-events?fromStartDate=DEBUT&toEndDate=FIN>)
- Nom (de l'appareil ou du technologiste) : De même ces champs viendront s'ajouter aux paramètres de la requête url envoyée au micro-service.
- Champs à afficher accompagnés des boutons de recherche. On pourra ici choisir parmi une liste de champs à afficher au moment des résultats de l'analyse. Les résultats s'afficheront alors en fonction des raisons de la non-acceptance, du mode d'acquisition ou de chaque technologiste.
Les boutons pourront lancer la recherche ou réinitialiser les champs du formulaire.

- **JavaScript**

Le fichier JavaScript aura un rôle de "backend" contrairement à l'HTML qui lui avait un rôle "frontEnd". Ainsi, c'est le script qui sera responsable de l'interaction avec l'utilisateur. Il jouera un rôle essentiel dans la mise à jour du formulaire, les requêtes https et l'analyse.

Requête

C'est le script qui se chargera de faire le lien avec le serveur via les requêtes https, celui-ci pourra ainsi la compléter avec les paramètres requis. Les requêtes seront au format 'GET' et prendront au minimum une période grâce aux arguments "startDate" et "endDate", auxquels pourront s'ajouter un appareil et un technologiste.

Le micro-service renverra comme réponse à la requête une liste vide ou composée d'RRAEvents. Le script utilisera donc uniquement ces listes pour son analyse et pour remplir certains champs du formulaire.

Formulaire

Pour la période, les champs sont initialement basés sur une période d'un mois, ainsi le script met par défaut la date de début à celle d'il y a un mois et la date de fin au moment présent, cela filtre donc la liste des RRAEvents saisis sur le dernier mois.

Pour les noms, le script fournit une liste des appareils et des technologistes présents dans la base de données, cela permet de s'assurer que les noms rentrés en paramètre dans la recherche correspondront bien à des noms existants.

Enfin, la liste des groupes à afficher ne dépend que du HTML puisque cette liste est fixe et n'est pas amenée à changer. Le bouton filtrer ("Show report"), va lancer et afficher l'analyse, tandis que le bouton reset remet chaque champ à sa valeur de base (ce bouton est en JavaScript et non le bouton reset utilisé en HTML car certains champs ont des valeurs non nulles).

Analyse

Une fois le bouton 'filtrer' sélectionné, le script envoie un message au serveur qui lui renvoie la liste des RRAEvents requis. Le script utilisera cette liste pour créer un bilan d'analyse en fonction des filtres demandés. L'analyse présentera ainsi plusieurs éléments de réponse :

Reason	Accepted	Repeated	Rejected	Non accepted percentage
BLANK	2	0	0	0%
CLINICAL_ARTIFACTS	0	0	0	0%
Total	2	0	0	Non accepted: 0%

- Un tableau comprenant le nombre d'image Acceptées - Rejetées et Répétées en fonction des technologistes/Raisons/Protocoles ... Accompagné des totaux de chacun. Pour cela, le script récupère pour chaque RRAEvent de la liste la sélection d'affichage (ici par Raison), et compte le nombre d'itérations qu'il affichera dans chaque case du tableau.
- Des graphes mettant en valeur le nombre de rejets et ses "causes". Pour cela, le script récupère les chiffres de la partie "Total" du tableau et utilise ces données pour en faire un graphique, celui-ci permet ainsi de mettre en avant les protocoles utilisés lors des rejets et les raisons de ceux-ci.



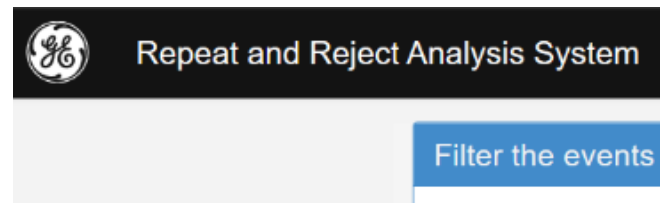
FIGURE 17 ; APERÇU D'UN RESULTAT D'ANALYSE RRA

- Enfin, si les résultats obtenus sont intéressants pour une analyse ils pourront être exporter au format PDF. Cependant cette fonctionnalité n'est pas encore implémentée si ce n'est son bouton, celle-ci est prévue pour la fin de mon stage et sera réalisée par le script.

Export at PDF format

- **CSS**

Pour finir, un fichier de style CSS (Cascading style sheets) a été ajouté afin de venir compléter le design de l'interface HTML et JavaScript. Le CSS permet d'uniformiser les styles et de rendre le rendu visuel beaucoup plus développé. Il contient par exemple un fichier CSS Bootstrap permettant l'amélioration du rendu visuel tel que des panels, des barres de navigations, des tableaux... De plus, l'équipe de développement a mis en place du code CSS permettant d'uniformiser les couleurs et les formes utilisées dans les pages HTML afin que celles-ci aient un rendu similaire. Grâce à cela, la page HTML que j'ai pu développer ne s'éloigne pas des autres pages produites auparavant. On retrouve ainsi dans la page, le design et les couleurs utilisées par GE Healthcare.



2) Outils et méthodes

A. Outils

a) Micro-Service

Description

Le serveur est un micro-service. Il fonctionne indépendamment des autres composants, gère une ressource spécifique (RRAevents) et peut ainsi être mis à disposition n'importe où. Il est d'ailleurs prévu de mettre ce service sur un docker pour une meilleure portabilité.

Le micro-service comprend une Interface permettant de définir la ressource standard, une partie application qui fait tourner le micro-service et une partie acceptance qui comprend les tests à réaliser pour vérifier le bon fonctionnement du service.

Une fois le développement du micro-service terminé, seuls quelques fichiers seront nécessaires pour son fonctionnement. On gardera ainsi le fichier .jar qui sera exécuté pour le lancement, des scripts permettant le démarrage et l'arrêt, et un fichier définissant les variables d'environnements définissant le chemin vers les fichiers requis par le micro service. Ce dernier permet de n'avoir que ce fichier à modifier lorsque l'on déplace le micro-service.

La Gateway suivait le même principe. Cependant celle-ci ne gérait pas de ressources, ce qui est une des caractéristiques principales d'un micro-service.

Les scripts

Il existe des procédés permettant de faciliter et d'automatiser la création de micro-services, ces procédés sont regroupés dans des exécutables (pour Windows avec des .bat ou pour Linux avec des .sh), une fois exécutés, ces scripts ont chacun une fonction apportant au développement ou au fonctionnement du micro-service.

- Openapigen : Permet de générer le code de l'interface à partir d'un fichier au format '.yaml' qui définit l'API et donc le format de la ressource standard.
- Build : Compile avant de pouvoir lancer le micro-service

- Run: Lance le micro-service et donc le fichier main de l'Application. Ecrit dans le fichier de logs pour obtenir les infos.
- Package: Crée un fichier .jar que l'on exportera pour micro-service final.
- Acceptance: Lance les tests d'acceptance.
- Check: Génère un rapport sur le code utilisé et son efficacité, grâce à l'outil "SonarQubes"
- Start: Démarre le micro-service
- Stop: Arrête le micro-service en tuant son process en cours.

Enfin, le fichier de script contient les fichiers de log, permettant de suivre la trace du micro service (message affiché lors du développement du micro-service par le développeur afin de suivre le bon fonctionnement de celui-ci). Il possède également le fichier de configuration contenant les variables d'environnements.

b) Technologies

En parallèle, le but du stage était d'utiliser de nouveaux outils mis récemment en place dans le service. Ce qui a permis au service de préparer le futur. Cela m'a également apporté une solide expérience. Voici un bilan des principaux outils utilisés.

Swagger Editor

Un éditeur pour code YAML
opensource, pour la définition et la
documentation d'API RESTful.

Il est disponible en ligne et permet une bonne visualisation du code ainsi que de la compréhension de celui-ci grâce à une bonne visualisation.

repeatRejectAnalysisEvents

Method	Endpoint	Description
GET	<code>/repeat-reject-analysis-events/count</code>	Count for Repeat and reject events
GET	<code>/repeat-reject-analysis-events</code>	Search for Repeat and reject events
POST	<code>/repeat-reject-analysis-events</code>	Create a new RepeatRejectAnalysis Event

RepeatRejectAnalysis.REPEATREJECTANALYSIS

Documents

Aggregations

Explain Plan

Indexes

FILTER

INSERT DOCUMENT

VIEW

LIST

TABLE

_id: "e55a492c-34a2-4ee5-affb-485a39a48fe0"

PROTOCOLNAME: "prot.test"

DATE: 6756172213046476800

STATUS: 1

✚ IMAGE: Array

✚ 0: Object

IRADIATIONEVENTUID: "1.2.840.10008.1.0.3.0.0.2.0.9.0.6.0.4.0.5.0.2.0"

ACCOUNTNUMBER: 0

MongoDB

MongoDB est une base de données NoSql, elle utilise un système de gestion de ses données avec un format proche de celui de JSON. Celui-ci est amélioré contrairement au SQL pour la gestion de l'orienté objet et à plus large volume, mais également pour l'utilisation de la méthode agile.

Maven

Apache Maven ou mvn, est un outil de gestion et d'automatisation de production des projets logiciels Java basé sur le concept des POM (Project Object Model). Il est utilisé par les différents script afin de faciliter l'intégration des composants dans le micro-service. Le POM permettra de saisir les dépendances du composant et permettra d'assurer la compilation du code de manière automatisée. Maven sera également utilisé ici pour l'auto génération de code à partir des IDL et des fichier YAML.

Cucumber

Est un outil développer afin d'automatiser les tests permettant la méthode BDD (behavior-drivent development) utilisée en Agile. Il permet de spécifier les tests attendus dans le langage courant afin que les "clients" puissent le comprendre. Son fonctionnement réside dans la définition d'étapes (appelées scénario) que le test doit réaliser. Ces étapes seront ensuite traduites dans le langage informatique pour les tests des logiciels. Voici un exemple simplifié d'un des scénarios réalisés durant mon stage, chaque ligne du scénario correspondra à une méthode de test. Ceux-ci seront réalisés les uns à la suite des autres et permettront une fois enchainés de valider chaque cas possible à tester pour le logiciel

Feature: Repeat and Reject Analysis Use cases

Scenario: Post a new rra event

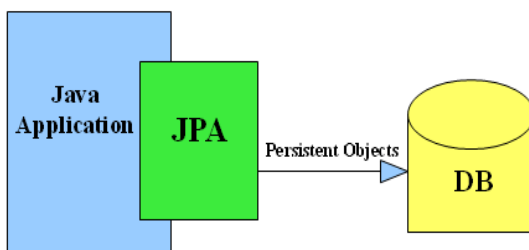
Given the micro-service is started

When posting a new "RRAEvent"

Then API response code is 201

And the "RRAEvent" is posted on the database

JPA



"Java Persistence API" permet de faire le lien entre l'Application d'une API et sa base de données. Ainsi, on codera les actions de la base de données en SQL et JPA se chargera de les faire correspondre à la base de données utilisée. Cela permet d'avoir une Application totalement dissociée de la base de données, et de pouvoir changer celle-ci à tout moment sans modifier le code de l'Application. Pour le moment, JPA a permis de faire le lien entre MongoDB et l'application du micro-service java.

SonarQubes

C'est un logiciel libre qui permet de vérifier la qualité du code analysée. Ainsi, une fois exécuté (via le script "check.bat"), Sonar va fournir un rapport indiquant le code pouvant être amélioré, étant répété, présentant des non conformités. Il validera aussi si le pourcentage de test couvert est suffisant et affichera le temps nécessaire estimé pour ces modifications ainsi que la différence par rapport au dernier test.



GitLab

Est une application web dites devOps, elle permet d'héberger et de gérer les projets 'Git'. Une interface utilisateur simplifie la gestion des versions du code de chaque équipe et de sa mise en commun.

P4

Permet d'effectuer un suivi de chaque modification des logiciels de l'équipe. Perforce est un outil de gestion de configuration que l'équipe utilise pour la mise en commun du code développer et le bon suivi de celui-ci.





Oracle VirtualBox

Les machines virtuelles ou "VM" sont utilisées afin de recréer sur un ordinateur, une seconde machine. VirtualBox est donc un logiciel émulateur qui simule une station d'acquisition afin que l'équipe puisse tester ses logiciels. La VM est ainsi capable de créer un faux examen de mammographie et d'acquérir des images. Cela permet d'appliquer les logiciels développés sur des situations quasi-réelles sans avoir à utiliser de vrais mammographes mais simplement son ordinateur.

Rti -DDS

Le logiciel Rti connect DDS (ou Data Distribution Service) est un protocole permettant la communication entre différents composants d'un ou plusieurs logiciels. Très utilisé pour l'utilisation d'API, DDS facilite le transfert de données en créant un « databus » sur lequel les différents composants vont pouvoir lire et écrire sans avoir à gérer les autres composants. Grâce à un format prédéfini, les composants pourront poster des données sur des "Topics". DDS se chargera alors de notifier les autres composants inscrite à ce même topic, qui pourront utiliser ces données par la suite. Dès lors, au lieu de devoir relier tous les composants les uns aux autres de manière archaïque, les composants s'ajoutent aux topics souhaités sur le databus. Ce qui facilite également l'ajout de nouveaux composants.

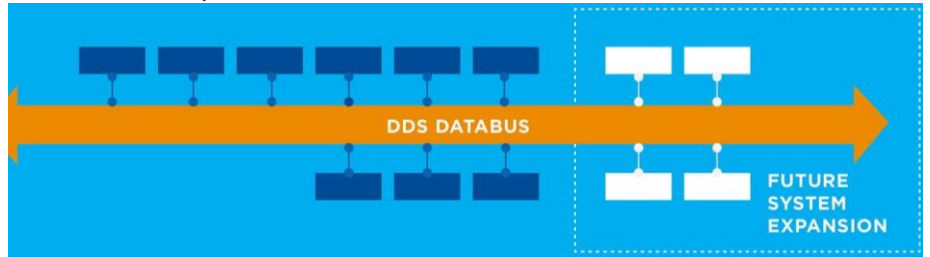


FIGURE 18 : TECHNOLOGIES UTILISEES

IDL

Est un langage de programmation (proche du C++) utilisé afin de formaliser les données complexes. Les IDLs sont notamment utilisés pour définir les données échangées dans le DDS databus. En effet, les composants inscrits à un Topic devront connaître le format de données qu'ils auront à traiter, formats définis dans chaque IDL. Pour finir, un RRAEvent possèdera son propre IDL, définissant toutes ces données (statut, date, etc...), ce qui permettra d'indiquer sur le Topic DDS le format à utiliser. Lorsque le mammographe postera un nouvel RRAEvent, il utilisera le format défini par l'IDL, format que la Gateway utilisera également.

Langages

Pour finir, j'ai eu la chance d'utiliser, en parallèle à de nombreux outils, de nombreux langages de programmation ce qui a été extrêmement formateur.

Tout d'abord, il a été décidé d'utiliser le JAVA pour le micro-service car celui-ci présentait des bibliothèques de méthodes plus développées dans le domaine des micro-services. En parallèle, j'ai pu travailler sur du YAML, tandis que cucumber ne nécessitait pas de langage particulier supplémentaire (si ce n'est l'anglais). La gateway suivant le principe du serveur et le langage utilisé était le même.

Les composants du mammographe étaient codés en C++ ce qui permettait une meilleure gestion de la mémoire allouée, et une meilleure précision, tandis que les composants DDS utilisaient des IDLs. Enfin, l'interface utilisateur d'image utilisait du QML pour les graphismes ainsi que du QT pour la gestion des événements via un design MVC.

B. Méthode

a) Planification et suivi

A l'image de l'équipe que j'ai pu intégrer, la méthode de développement que j'ai utilisée a été la méthode Agile. Il a donc été prévu de réaliser un suivi de stage étape par étape utilisant ce modèle. Nous avons ainsi défini avec mon tuteur chaque étape à remplir en User Stories, puis en les répartissant dans un ordre selon les prérequis. Ainsi, j'avais à disposition ce tableau, indiquant les différents besoins à remplir et l'ordre de chacun.

En parallèle, la réunion quotidienne lors du "stand up", permettait de suivre mon avancée. Pour cela, chacune des tâches décrite dans les User Stories avaient été redécoupées en plus petites tâches. Celles-ci étaient réalisables en une semaine environ et montraient ainsi l'avancée détaillée de mon stage.

b) Direction

Voici le tableau utilisé pour décrire les user stories de mon stage.

Steps	Titile	Acceptance criteria	Done
1	Create RRA (μSvc)	RRA μSvc compile, run	Accepted
2	Add POST to API	Possibility to add an RRA event through Postman	Accepted
3	Add GET to API	Possibility to get a list of events (rra) sorts by date	Accepted
4	Report UI Creation	Demo: From Post (Postman) to the page display	Accepted
5	finalization of RRA μSvc	Valid Unitary Tests and acceptance tests, coverage over 90%, Confluence explanation of the Service and the advancement, Cucumber demo.	Accepted
6	RRAREST Gateway creation	Post an RRA event on DDS with the Gateway translation. Then, the RRA Event is created on the μSvc.	Accepted
7	ExamManager adaptation to write the RRAEvent on DDS	Functional SATI test of ExamManager, when receiving a DataSetElement, posting an RRAEvent	Completed
8	AcquisitionViewer: Add the User Interface	Functionnal SATI / Squish test: When pressing the button => Writing a DataSetElementState Demo : from the user click to the UI report	In progress
9	PDF of RRA result creation	validated by the Product Owner	define
11	XRAY API modification	GET /repeat-reject-analysis-events/count DELETE /repeat-reject-analysis-event/{id} DELETE /repeat-reject-analysis-events GET /repeat-reject-analysis-events/ with limit and offset GET /repeat-reject-analysis-event/{id} with 401	define

Chacune de ces stories possède un état. On retrouve cet état sur un tableau présent lors du StandUp permettant ainsi de suivre les stories :

Défini

Signifie que toute la User Story a été définie et répartie en plusieurs petites tâches, le sujet à faire est bien compris. Pour cela, j'avais des réunions au préalable avec mon tuteur de stage ou un des architectes afin d'expliquer et d'assurer la compréhension du sujet mais également de l'architecture et des moyens de conception de celui-ci.

En cours

Lorsque l'on commence à travailler sur une tâche, celle-ci passe alors à l'état « en cours ». Durant cette étape, j'ai pu bénéficier de l'aide de nombreux collègues et de mon tuteur de stage aux moments d'incompréhensions face à différents problèmes. Les réunions du Stand Up permettaient de suivre le niveau d'avancée de la tâche, grâce aux explications et aux découpes en plus petits objectifs, ce qui apportait plus de précision.

En revu

Une fois le composant ou le code terminé, celui-ci était envoyé en revue à un membre de l'équipe et à mon tuteur de stage, afin qu'il puisse m'indiquer de potentielles erreurs ou des améliorations possibles, voire même, des commentaires à rajouter. Cette revue de code se faisait via l'outil Git, et permettait d'avoir un code optimal.

Compété

Afin de s'assurer d'avoir bien complété les objectifs de chaque partie du stage, nous avons fixé au préalable des conditions de validations. Celles-ci devaient être vérifiées afin de pouvoir passer à l'étape suivante. Cela donnait ainsi une bonne idée de ce qu'il fallait accomplir et de ma manière de réaliser les tests de chacune des étapes. Par exemple, la réalisation de testes unitaires sur toutes les méthodes "privées" permettait de s'assurer que celles-ci réalisaient la bonne action. Un test unitaire consiste à appeler une méthode du code produit et de vérifier que celle-ci renvoie bien ce qui était prévu, cela permet de s'assurer du bon fonctionnement de la totalité des travaux. De même, la réalisation des tests d'acceptances était également primordiale pour la validation de cette étape.

Accepté

Pour valider une étape du stage, et donc du planning Agile, il fallait en plus des tests et des revues de codes, que mon tuteur (ou un architecte) valide le composant afin que celui-ci devienne accepté. Pour cela, il vérifiait d'abord le fonctionnement général de ce composant, puis, il s'assurait également que les objectifs qui avaient été décrits dans la définition de la story avaient bien été remplis. En temps normal, le Product Owner de l'équipe Agile est également amené à apporter sa validation.

Enfin, tous les composants étaient livrés au fur et à mesure sur l'outil Git, assurant ainsi la sauvegarde de ceux-ci. Une fois les composants destinés, on pouvait les livrer sur une branche différente de l'habituelle, marquant ainsi la fin d'une première version.

3) Résultats

A. Prévisions

a) Rappel des objectifs

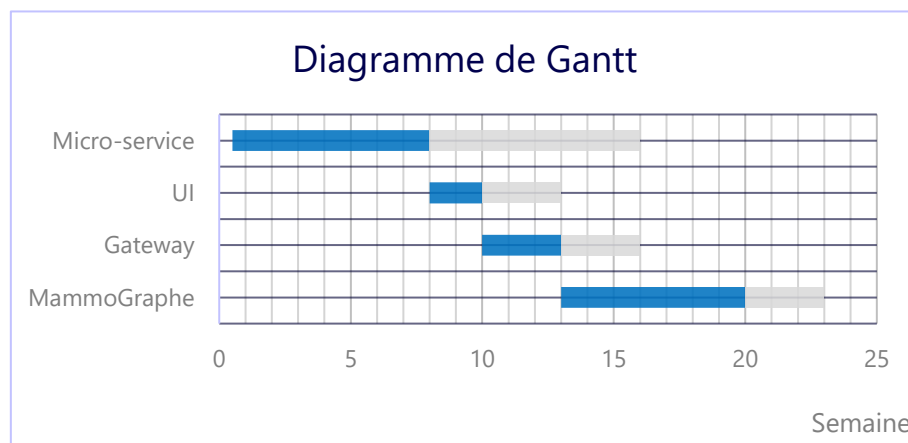
Dès le début nous avons fixé mon tuteur et moi, l'objectif d'avoir l'ensemble des composants utilisables et présentables. Puis, en fonction du temps restant, d'ajouter les besoins non essentiels au service. Pour cela, nous avons fixé l'ordre nécessaire des composants selon leurs dépendances. Ainsi, il a été choisi de commencer par la réalisation du micro-service. Celui-ci permet en effet de définir la structure de données à utiliser pour chaque composant et c'est lui qui va stocker ces mêmes données. Il est donc la base de ce projet car tous les composants vont chercher à communiquer avec lui. Nous avons ensuite décidé de développer l'interface utilisateur afin d'avoir enfin un vrai rendu visuel du travail effectué jusque-là, permettant ainsi de voir concrètement les données utilisées et ce pour quoi elles allaient être utilisées.

De nombreux autres demandes ont été ajoutées pour le produit final, mais ces requêtes n'étant pas primordiales, celles-ci sont passées après les besoins exprimés ci-dessus.

b) Diagramme de Gantt

Le diagramme de Gantt représente le temps passé sur les différentes parties du stage et leur enchaînement.

Il décrit également la durée des modifications suivant la "fin" d'un des sujets. En effet, les revues de codes ou l'apport de nouvelles problématiques, ont fait tel qu'il a fallu revenir sur ces sujets afin de les améliorer selon certains besoins. En effet, lors de projets logiciels, il y a souvent un besoin d'améliorer le travail fourni et de le modifier selon de nouveaux besoins.



Dès lors, la première semaine de mon stage j'ai plus été occupée par l'installation des outils nécessaires ainsi que de la compréhension de mon sujet et des travaux déjà réalisés dans l'équipe.

- ❖ Par la suite, j'ai pu travailler sur la mise en place du micro-service. C'est la partie la plus longue puisqu'en plus d'être technique et composée de technologies nouvelles, il a fallu mettre en place le format de data que j'allais utiliser tout au long du stage et comme donnée standard pour le micro-service à savoir un "RRAEvent". Ce format sera quasiment le même pour les autres composants, son traitement sera donc bien moins long.
- ❖ Puis, j'ai pu commencer en parallèle l'interface utilisateur. Celle-ci a été plus rapide car elle comprenait des langages plus simples sur lesquels j'avais déjà pu travailler le semestre précédent (HTML et JS).

C'était de plus l'analyse du travail réalisé précédemment et motivé par le fait de pouvoir enfin voir le résultat visuel de cette première partie du stage.

- ❖ Une fois l'UI et le micro-service complétés, j'ai pu commencer l'envoi de données au serveur via la Gateway. Ce composant a également été plus court car il consistait en la récupération d'une donnée et son envoi au micro-service. La partie plus compliquée aura été de comprendre le fonctionnement de DDS et des IDL, outils que je ne connaissais pas encore.

Il faut préciser qu'à ce niveau-là, un changement de la structure des données de la Gateway entraînait un changement dans celle du micro-service et inversement. Ainsi les dernières modifications de la Gateway et du micro-services ont eu lieu au même moment, d'où leur fin commune sur le diagramme.

- ❖ Enfin, je finis au moment de l'écriture du rapport le dernier composant du sujet. Il me semble que celui-ci est le plus long et compliqué. Il faut en effet modifier le mammographe alors que le travail apporté sur les logiciels de celui-ci est déjà énorme. Il faut en effet pour modifier chaque composant, comprendre auparavant les codes, l'architecture et les bonnes pratiques déjà mis en place par l'équipe. De même, ce travail s'effectue sur une machine virtuelle. En effet, chaque modification du logiciel doit ensuite être mise sur la machine virtuelle, ce qui est plus long que simplement tester sur sa machine de développement. Pour finir, les technologies utilisées ici (C++, QML, Qt) étaient nouvelles pour moi, ou du moins peu approfondies ce qui joue aussi sur le temps passé à répondre aux besoins.

Pour finir, une fois l'étape de la modification du mammographe terminée, il restera de plus petites tâches de finalisation. Celles-ci correspondront aux dernières modifications nécessaires et aux besoins plus compliqués et moins importants qui avaient été gardés pour la fin du stage.

Finalement, ce diagramme permet de bien refléter le temps passé sur chaque composant du projet. Il faudra cependant bien prendre en compte que derrière celui-ci se trouve de nombreuses modifications et améliorations apportées par la suite, mais également, un temps important passé sur des mêmes problèmes (de résolution de bug, de compréhension de la structure mise en place, de correction apportée par la suite et de découvertes de nouveaux moyens de résolution). Je pense que c'est ce qui a enrichi le plus mon expérience professionnelle : apprendre à chercher seul la solution à son problème. En effet, à de nombreuses reprises, j'avais tendance face à cela, à attendre de l'aide de la part de mes collègues plus expérimentés dans ces sujets. Cependant, au fil du temps et en les observants résoudre des questions auxquelles ils n'avaient parfois eux même pas les réponses. J'ai appris à mieux chercher (sur internet ou alors dans les fichiers d'architecture et de logs), me permettant d'acquérir plus d'autonomie et de bien faire la distinction quand de l'aide était nécessaire ou non, car cela reste important de bien communiquer afin de ne pas partir dans une mauvaise direction.

c) Modification des prévisions

Plusieurs types de changement ont pu être apportés tout au long du projet.

Changements apportés

Dans un premier temps, des changements ont été apportés au niveau du design et de l'architecture. Le projet étant pour le futur et non dans les priorités de l'entreprise pour le moment, l'architecture a bien été réalisée mais n'était pas figée. C'est d'ailleurs le cas sur la plupart des projets informatiques où certains points se modifient durant le développement du composant pour différentes causes. Dans mon cas, les changements d'outils ont constitué les plus grosses modifications.

Le fait de modifier l'interface d'API entraîne un changement au niveau de tous les composants. En effet, cette interface définit la ressource utilisée comme modèle dans chacune des parties des services. Ainsi, ce changement de modèle entraînera une redéfinition de la ressource et donc une utilisation différente de celle-ci.

Il y a ainsi eu des changements au niveau du design de l'API, notamment au début lors de la définition de celui-ci. Cependant, une fois complétée, l'irradiationEventUID a remplacé un ancien attribut "imageSOPInstanceUID". Ce changement est dû à une différente conception de l'utilisation du service d'analyse. De même, afin d'apporter des informations supplémentaires, protocolName et imageType ont été ajoutés à la structure du RRAEvent.

De plus, l'utilisation de l'outil JPA ne s'est décidé que durant le stage et est donc venu remplacer complètement la persistance de l'application du micro-service. Ces changements sont plus longs sur le court terme mais sont essentiels pour le bon maintien du service. Par ailleurs, l'architecture de chaque composant est conçue afin de pouvoir accueillir facilement tout type de changement. C'est d'ailleurs l'avantage principal de JPA, d'où l'utilité de ces changements pour le bon maintien du produit.

Changements futurs

Dans un second temps, des changements ont été prévus une fois l'objectif principal atteint. Ces changements ont été ajoutés parmi les Users Stories à la fin du tableau des tâches. Ils sont importants, mais il a fallu prioriser les besoins pour s'assurer d'avoir un produit prêt avant la fin du stage, alors que les éventuels changements pourront être apportés par la suite.

Le plus gros changement prévu est l'étendue du micro-service "serveur" de l'analyse des images répétées et rejetées. En effet, l'équipe de développement située en Inde, travaillant avec mon équipe, a pour projet d'apporter ce système d'analyse sur les machines à rayon X pour le vasculaire. L'idée serait donc de modifier l'interface de l'API afin que celle-ci soit compatible avec les deux systèmes. Cependant, ces besoins ne sont pas encore assez clairs pour les apporter au micro-service existant, et des discussions avec l'équipe seront nécessaires avant cette mise en place.

B. Résultats

a) Descriptions

Au moment de la rédaction du rapport, je finis la modification du mammographe afin de créer un ou plusieurs RRAEvents. Le micro-service RRAServer et la Gateway sont complets, tandis que l'interface utilisateur pourra être amélioré même si la version actuelle est complète et utilisable.

J'ai pu réaliser une première étape que j'ai présentée à l'équipe :

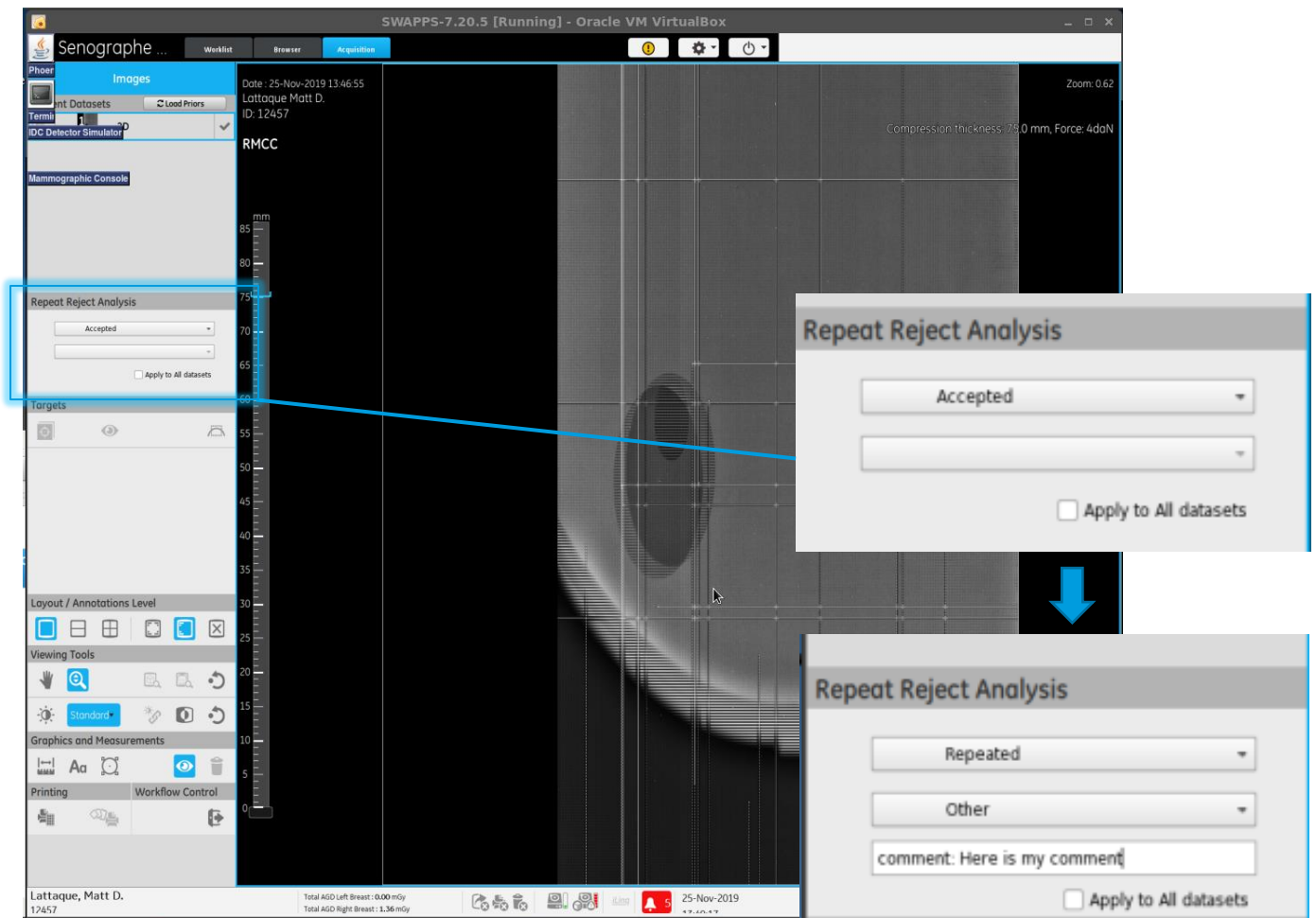
- 1) A la fermeture d'un examen, le mammographe poste un message DDS pour chacune des acquisitions, avec les infos RRA disponibles (non spécifique à ce service tel que les noms d'appareil, de technologiste, types d'images, identifiants de celles-ci) et les informations du RRAEvent (le statut, la raison en cas de non-acceptance et les commentaires potentiellement rajoutés) sont directement déclarés dans le code "en dur" pour le moment. Les RRAEvents postés ont donc pour le moment (et par défaut) un statut "d'accepté" pour le moment avec un commentaire vide et la raison à "BLANK".

- 2) Ce message est récupéré par la Gateway et envoyé au micro-service qui stockera chaque event.
Toujours, pour les tests, la Gateway est placée sous la machine virtuelle avec le système de simulation du Mammographe
- 3) On peut observer l'impact des messages sur l'analyse présentée par l'interface utilisateur.

Cette première étape, permet d'observer l'avancée du travail de manière concrète et représente une certaine récompense du travail fourni. Ainsi, ce premier résultat représente une certaine satisfaction.

b) Aperçu

Dès lors, voici le rendu visuel :



On lance une simulation de pristina sur la machine virtuelle, une fois l'acquisition simulée, on obtient une image, on va alors sélectionner le statut « Répétée » ce qui va nous permettre d'entrer une raison, en sélectionnant la raison « Other », le champ de commentaire va apparaître.

Avant de fermer l'examen, on ouvre l'outil « rtiadminconsole » et on va se placer sur le topic rraEvent.

Sur celui-ci, on peut voir que trois composant sont liés à ce sujet :

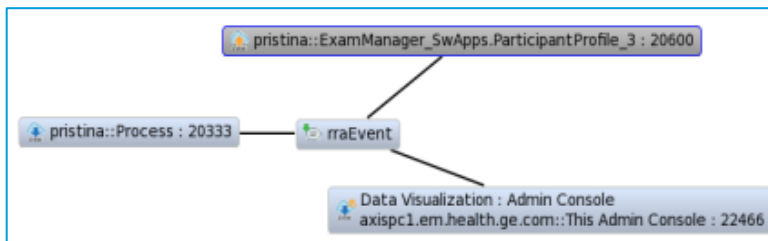


FIGURE 19 : CAPTURES D'ECRAN DES RESULTATS SUR LA MACHINE VIRTUELLE DANS ACQUISITION VIEWER

- (en haut) ExamManager qui va poster des RRAEvents.
- (à gauche) la Gateway qui va récupérer ces messages postés une fois lancée.
- (en bas) le visualiseur de données de l'outil RTI qui permet de lire les messages postés.

Parallèlement, on a démarré le micro-service via le script "Run.bat" comme ci-dessous.

```

Stage
  "MS Run"
Running...
Loaded V:\mammo-windows-dev-station\projects\checkout\rra-micro-service\script\config\env-dev.vars
[INFO] Scanning for projects...
[INFO] -----< com.gehc.mammo.rams:rrams >-----
[INFO] Building rrams 0.1.0
[INFO] -----[ jar ]-----
[INFO] --- exec-maven-plugin:1.6.0:java (default-cli) @ rrams ---
Application logs in V:\mammo-windows-dev-station\projects\checkout\rra-micro-service\script\logs

```

Une fois le message reçu par la Gateway, celle-ci va alors envoyer si elle est bien configurée, un message http au micro-service. La Gateway peut être sur un appareil différent du serveur car ceux-ci communiquent via REST. C'est d'ailleurs le cas ici puisque le micro-service est sur un appareil Windows tandis que la Gateway fonctionne dans la machine Virtuelle, elle-même sous un appareil utilisant Linux.

On peut voir dans la trace du micro-service que le micro-service a bien reçu le message de la Gateway et que celui-ci l'a enregistré via JPA dans la base de données.

```

[INFO] [com.gehc.mammo.rams.persistence.JPAPersistenceManagerImpl save] Save entity: com.gehc.mammo.rams.business.model.RepeatRejectAnalysis@57585dfc
[INFO] [com.gehc.mammo.rams.persistence.JPAPersistenceManagerImpl save] Save entity: com.gehc.mammo.rams.business.model.RepeatRejectAnalysis@292364e6
[INFO] [com.gehc.mammo.rams.persistence.JPAPersistenceManagerImpl save] Save entity: com.gehc.mammo.rams.business.model.RepeatRejectAnalysis@39d1a1ca
[INFO] [com.gehc.mammo.rams.persistence.JPAPersistenceManagerImpl save] Save entity: com.gehc.mammo.rams.business.model.RepeatRejectAnalysis@6c775c91

```

FIGURE 20 : CAPTURES D'ECRAN DU MICRO-SERVICE (SON SCRIPT, SA TRACE ET SA BASE DE DONNEES)

Lorsque l'on se connecte sur le logiciel de gestion de MongoDB, on peut alors voir la liste des objets postés sur cette base. On peut ainsi voir le dernier objet de la liste, correspondant à l'objet que l'on vient d'acquérir. Celui-ci contient toutes les informations utiles au format de MongoDB qui seront traduits à sa sortie par JPA.

Par la suite, on pourra observer les résultats acquis en filtrant la recherche en fonction des technologistes et de l'appareil au moment l'acquisition. Ci-dessous le résultat graphique d'une recherche pour un RRAEvent posté lors d'un test.

```

{
  "_id": "6ecf9e6a-c81f-4cd7-b9c9-0f7330466512"
  "PROTOCOLNAME": "PROCEDURE_ROUTINE_2D_DUAL_ENERGY"
  "DATE": 6761434982503153664
  "STATUS": 2
  "IMAGE": Array
    0: Object
      "IRRADIATIONEVENTUID": "1.2.840.10008.1.1.2.1.0.8.3.6"
  "ACQUISITIONMODE": 0
  "TECHNOLOGIST": Array
    0: Object
      "TECHNOLOGISTNAME": "Dupont"
  "OPTIONALCOMMENT": "Leave any optional comment here"
  "DEVICE": Array
    0: Object
      "STATIONNAME": "Deus"
  "IMAGETYPE": "oui"
  "REASON": 9
  "CLINICALVIEW": "RMCCID"
}

```

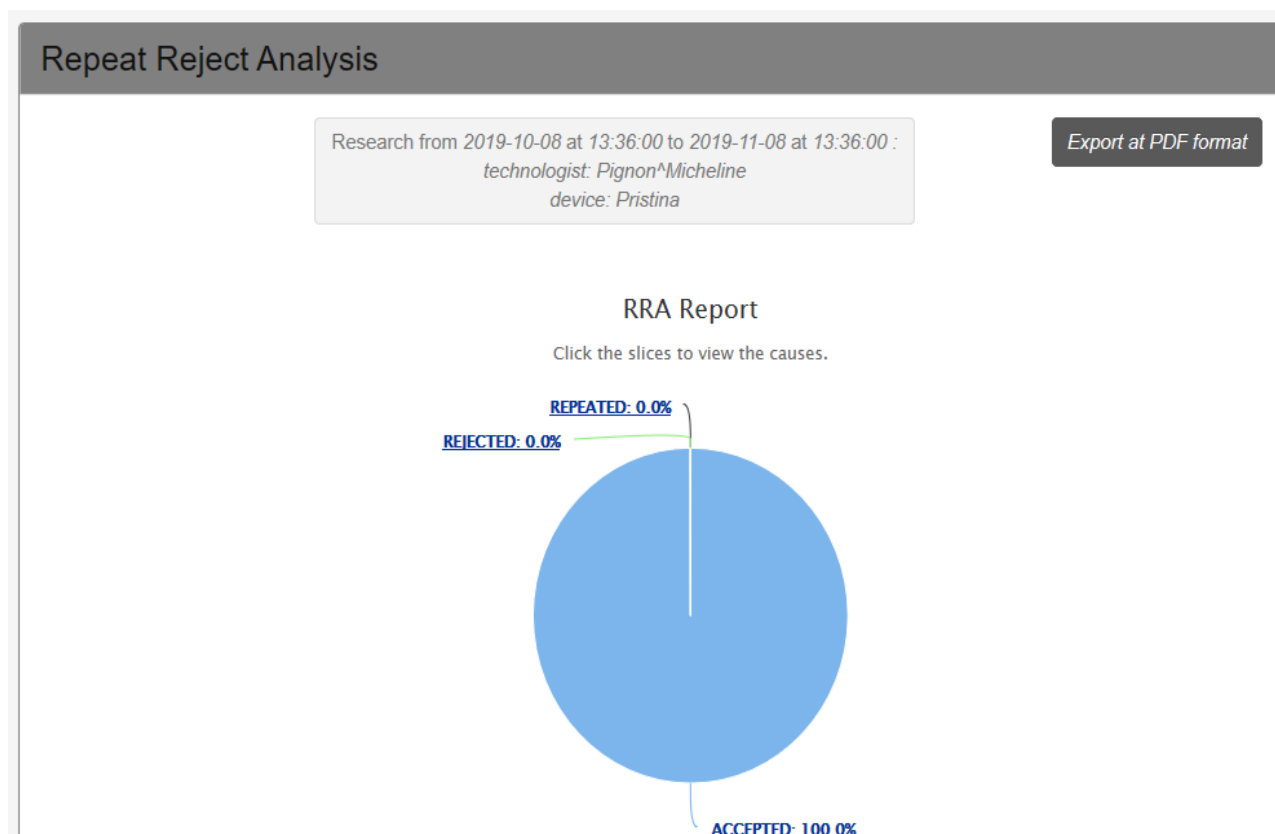



FIGURE 21 : RESULTAT GRAPHIQUE CLIQUABLE D'UNE ANALYSE

c) Objectifs restants

Tous les composants de l'objectif sont donc présents dans la démonstration. Cependant, la partie du mammographe n'est pas encore complétée. En effet, il faut désormais pouvoir saisir sur celui-ci les informations requises pour RRA particulièrement (Status, Raison, commentaire) qui jusque-là n'existaient pas. Les boutons de l'interface utilisés pour remplir ces champs ont été ajoutés. Il faut désormais créer les informations via les champs remplis par l'utilisateur. Pour cela, le design Model-View-Contrôleur doit être modifié, et celui-ci utilise également des messages DDS afin de communiquer entre la vue et le modèle (entre AcquisitionViewer et ExamManager).

Enfin, de nombreuses tâches ont été repoussées à la fin du stage celles-ci n'étant pas prioritaires. Il sera donc intéressant de voir mon avancée sur cette période. Je souhaiterais bien évidemment pouvoir arriver à une version utilisable qui pourra être déposée sur un rendu officiel de l'équipe.

C. Analyse

a) Discussion des résultats

Avancée et planning

Tout d'abord, je suis plutôt satisfait de l'avancée du projet. J'approche de la fin de l'objectif principal à savoir pouvoir remplir et faire transiter un RRAEvent du mammographe jusqu'à l'analyse. Cependant, je pense que ma progression aurait pu être plus importante. Ainsi, ma vitesse de production aurait pu être améliorée (sans pour autant réduire la qualité), si j'avais été plus efficace dans la détection des bugs et des

erreurs de code. Je reste cependant conscient que cela fait partie du métier que de passer plus de temps à corriger les erreurs que de produire du code, d'autant que la compétence à fixer une erreur vient au fur et à mesure avec l'expérience. J'ai notamment pu constater avec le recul une meilleure gestion des erreurs au fil de mon avancée tout au long du stage. Pour finir sur mon avancée, je pense donc que celle-ci est correcte au vu de mon expérience. Je présente cependant un regret au vue du temps passé sur la correction. J'estimerais approximativement mon temps passé à produire 75% du code, à 25% de mon temps ; pour 75% du temps à corriger les 25% restants, ce qui se rapproche de ce que m'avait dit un collègue en plaisantant sur le métier de développeur.

Planning et objectifs

N'ayant pas d'échéance précise ou de deadlines, il est dur de juger ma propre avancée d'un œil extérieur. Je pense cependant, au vu des objectifs définis, avoir rempli une grande partie des tâches d'ici la fin de mon stage. En conséquence, les objectifs principaux devraient être très probablement atteints (Ceux-ci avaient en effet été définis de sorte à être facilement réalisables). En revanche, la réalisation de la totalité des objectifs restera très compliquée. D'autant que certains restent à définir plus précisément à l'image de la partie XRay à réaliser une fois le micro-service pour mammographe terminé.

De plus, durant la réalisation de cette première étape j'ai fait face à des problèmes que je n'ai pas su résoudre. Ainsi, il a été choisi de contourner ces problèmes en utilisant une autre méthode. Cela m'a permis de continuer à avancer sans rester bloqué longtemps. Cependant, cela n'a pas résolu le problème pour autant. Il sera donc bien de revenir sur ces points d'ici la fin du stage afin de trouver une solution et proposer un rendu final "propre" même s'il est parfois plus intéressant de changer de manière de faire que de rester bloqué trop de temps.

Finalement, les résultats à un mois et demi de la fin de mon stage sont satisfaisants, et j'approche de la fin du principal objectif. Cependant, le stage est loin d'être terminé, et j'espère pouvoir avancer jusqu'à la modification de l'API du micro-service pour les services X-RAYS, les tâches restantes suite à cela étant plus courtes.

b) Points de vue personnel

Apports techniques

Ce stage m'a permis d'énormément apprendre d'un point de vue technique, grâce à une pratique importante notamment.

Applications

- Dépasser la pratique

Dans un premier temps, ce stage a été pour moi non pas seulement l'occasion de mettre en pratique ce que j'avais appris durant mon cursus, mais également d'augmenter considérablement les acquis que j'avais en termes de connaissances techniques. En effet, j'avais une certaine appréhension en débutant le stage, de manquer de connaissances techniques face à certains langages ou outils informatiques n'ayant eu qu'une année de branche informatique. Cependant, j'ai rapidement pu combler ce manque grâce aux conseils de mes collègues de travail ou encore des réponses apportées sur internet. Je me rends ainsi compte que l'expérience et le temps passé sur l'utilisation complète la théorie et les projets d'écoles. J'ai ainsi pu développer largement mes connaissances techniques, prenant ainsi compte de l'expérience dans ce domaine (d'où la différence entre les ingénieurs logiciels juniors et seniors). J'ai également réalisé que même une fois diplômé, il me restera énormément à apprendre.

- Développer la résolution

De plus, le fait de travailler en autonomie sur un projet nouveau, m'a permis de sortir des habitudes que j'avais pu avoir lors des projets d'études. Ainsi, ce n'était plus des sujets comprenant déjà une correction ou une solution, ni des camarades qui avaient pu trouver une solution face au même problème auparavant. Ainsi, en cas de difficulté, il a fallu apprendre à chercher par soi-même, il n'était plus question de demander de l'aide à un professeur mais d'ajouter soi-même une trace conséquente dans le code sous faute de ne pas avancer de la journée. J'ai pu constater cela en observant mes réactions face à un problème au début de mon stage et maintenant, même s'il m'arrive évidemment de demander de l'aide à mes collègues, me permettant également de ne pas aller dans une mauvaise direction.

- Apprendre (toujours) de nouvelles choses

Enfin, j'ai eu la chance de découvrir et d'utiliser de nouveaux outils et langages tout au long de mon stage. J'appréhendais au début, le fait de ne pas connaître et de ne pas avoir l'expérience allant avec ces technologies. Je craignais ainsi de ne pas pouvoir avancer à cause de cela. Cependant, je pense avoir pu rapidement m'adapter grâce à la documentation présente sur internet et fournie par l'équipe. De plus, j'ai réalisé que je n'étais parfois pas le seul à ne pas connaître certaines technologies. En effet, et surtout en informatique, les technologies utilisées évoluent sans cesse, un ingénieur doit donc toujours se tenir au courant de celles-ci et réussir à adopter les technologies les plus efficaces. La capacité d'adaptation et de compréhension dans ce domaine sont donc essentielles.

Finalement, je pense qu'être ingénieur consiste à savoir apprendre, et qu'il est plus important de savoir se débrouiller face à l'inconnu plutôt que de maîtriser parfaitement un domaine sans en sortir. Bien entendu, la maîtrise a également un rôle essentiel.

Connaissances en micro-service

Dans un second temps, j'ai pu me rendre compte de l'avenir que représentent les micro-services. Je ne connaissais que très peu à mon arrivée, mais les micro-services représentent une véritable technique d'architecture pour le futur. Celle-ci permet en effet une véritable indépendance entre chaque composant, ce qui représente un véritable avantage pour l'utilisation de logiciels volumineux. C'est d'ailleurs le cas dans mon équipe de développement, qui a pour projet de migrer de nombreux composants sur des micro-services améliorant ainsi la mise à jour, le maintien, les tests et l'adaptabilité de ces composants.

De plus, le micro-service permet au développeur d'utiliser différentes technologies, ce qui représente un avantage énorme. Je me suis d'ailleurs rendu compte que les préférences de langages et de technologies variaient énormément d'un développeur à un autre. Le fait de rendre un composant simplement dépendant du format de donnée à utiliser et non de tout le reste peut alors être extrêmement intéressant pour une société et ses ingénieurs.

Je pense donc que le fait d'avoir travaillé du début à la fin de la création sur une telle architecture, représente un atout énorme pour mes connaissances, Être familiarisé avec une telle architecture avant même la sortie d'école représentera pour moi une vraie chance dans le futur.

De même, les outils utilisés pour ce micro-service sont pour la plupart assez récents, je pense donc que le fait de les découvrir dès maintenant me sera grandement utile face à la popularité des micro-services et me permettra peut-être même d'apporter quelque chose en plus dans ce domaine. Bien évidemment, il

est important de souligner que cette architecture, comme toutes les autres présente également des défauts (comme la complexité pour un simple service par exemple).

Développement et équipe

Dans un troisième temps, j'ai dû travailler en équipe sur un gros projet logiciel déjà existant. En effet, la plupart des projets logiciels sur lesquels j'avais travaillé n'avait pas un centième de l'ampleur du logiciel utilisé sur le mammographe Pristina. Des Ingénieurs Softwares expérimentés travaillent dessus tous les jours depuis des années, tandis que mon projet le plus long avait duré 6 mois sur un groupe de 6 personnes en première année de branche et donc avec des connaissances très limitées.

Cela m'a donc permis de mieux saisir l'impact d'une bonne documentation, du respect des standards, du travail en conférence et des outils tels que Git ou perfore. De même, j'ai pu prendre conscience de l'importance de la gestion des tâches dans une équipe et entre les équipes et du rôle des personnes chargées de s'assurer du bon déroulement des projets.

De plus, j'ai dû apprendre tout une partie d'Intégration informatique à laquelle je n'étais pas forcément habitué, et le fait d'avoir été proche de l'équipe d'intégration m'a permis je pense de saisir l'importance de celle-ci (essentielle à l'équipe si celle-ci veut appliquer son logiciel au produit). Cela m'a ainsi permis de me perfectionner dans cette partie du développement informatique, même si celle-ci reste encore parfois compliquée pour moi.

Pour finir, le fait de travailler sur un si gros projet m'a vraiment fait réaliser l'importance de détails que je ne saisisais pas forcément pleinement :

- En informatique à travers le besoin de coder pour soi au présent, mais aussi pour les autres dans le futur (laisser du code compréhensible et cohérent facilement modifiable).
- Dans une équipe de développement, en découvrant l'importance des rôles des personnes chargées d'assurer une bonne communication (scrum master, manager, product owner, personnes liées au produit moins directement). Cette impression vient compléter mon impression du stage ST05 où j'avais alors ressenti des problèmes dûs à une simple incompréhension entre deux services.
- En parallèle, j'ai eu l'impression que les ingénieurs softwares avaient peu d'influence sur les décisions du produit alors qu'ils sont au plus proche de la conception informatique de celui-ci.

Finalement, ce stage a présenté pour moi un apport technique auquel je n'aurais pas pu autant approfondir selon moi de manière scolaire. Le fait d'être dans une importante entreprise sur un sujet de développement pour les plans futur de celle-ci, a également représenté une chance pour moi et me distinguera sûrement d'autres étudiants. Enfin, j'ai pu observer l'importance de la communication, des problèmes que son absence peut occasionner, et j'espère ne pas oublier cette observation pour le futur.

Apports personnels

Expérience

Ce stage, en plus d'être pour moi l'occasion de gagner en expérience, m'a appris à mieux me connaître dans ce milieu. En effet, j'ai pu constater les points dans lesquels j'avais plus de facilités, et inversement,

les points plus compliqués pour moi. Il a également été pour moi l'occasion de voir mon ressenti vis-à-vis de différentes tâches.

Points faibles :

Durant ce stage, un de mes plus gros points faible a été mon manque de connaissance et d'expérience, cela reste normal pour un stage, mais j'espère pouvoir vite combler cela. En effet, je ne pense pas pouvoir avoir le niveau de connaissances qu'ont mes actuellement mes collègues à l'obtention de mon diplômes. Je pense donc que la différence entre mes futurs collègues et moi se ressentira tout de même au début de ma carrière contrairement à l'image que j'avais pu me faire d'un jeune diplômé. J'espère bien évidemment me tromper et pouvoir surprendre ma future équipe cela dit !

De plus, j'ai parfois présenté des lacunes dans les tâches d'intégration logiciel. En effet, le fait de devoir travailler avec des versions de code qui changent régulièrement, de mettre à jour des fichiers, mis à jour par d'autres personnes au même moment et de sauver les bons fichiers aux bons endroits a parfois été un problème pour moi. Le nombre de personnes présentes sur ces travaux étant conséquent, l'équipe d'Intégration doit sans cesse réaliser un gros travail de maintien à jour du logiciel, et les démarches à faire pour modifier ou récupérer celui-ci m'a parfois posé problème. J'espère pouvoir mieux intégrer cela lors de mes prochaines expériences, et désormais, plus rapidement.

D'un point de vue plus personnel, j'ai eu l'impression de parfois manquer de patience (surtout au début de mon stage), et de demander trop facilement de l'aide. Je pense commencer à m'améliorer sur ce point. Cependant, face à des erreurs non habituelles ou des éléments de codes et de fichiers que je ne connaissais pas, j'avais tendance à ne pas assez faire de démarches de recherche et de compréhension. Alors qu'une fois mes collègues présents, ces démarches paraissaient plus simples. J'ai grâce à cela pu les observer dans la résolution (lorsqu'ils ne comprenaient pas non plus), ce qui m'a permis de mieux saisir la méthode d'analyse d'erreur ou de compréhension de fichiers. Il en était de même lorsque j'utilisais de nouveaux outils. En revanche, il m'est également arrivé de vouloir comprendre sans trop d'aide et de partir sur de mauvaises pistes. Finalement, je pense qu'il est important de chercher à comprendre, mais qu'il faut aussi s'assurer de suivre la bonne démarche en s'informant auprès des autres. Il faut ainsi persévérer en communiquant efficacement et au bon moment.

Points forts :

A l'inverse, j'ai également pu constater des domaines dans lesquels j'avais des facilités. Tout d'abord au vue d'un travail beaucoup basé sur l'autonomie, je pense avoir réussi à montrer un côté assez débrouillard.

De même, le fait d'utiliser de nombreuses technologies nouvelles a confirmé une bonne capacité d'adaptation et une facilité dans l'apprentissage de nouvelles choses. Je me suis ainsi retrouvé à plusieurs reprises lancé sur de nouveaux outils avec pour exemple de la documentation ou d'autres codes existants, et j'ai eu l'impression de comprendre sans trop de soucis les attentes, la manière de faire et les bons procédés à utiliser. J'ai d'ailleurs apprécié me lancer dans de nouvelles technologies et découvrir celles-ci, je pense ainsi avoir une bonne adaptabilité sur les outils et langages utilisés.

De plus, j'ai beaucoup apprécié la partie conception et développement du logiciel, à savoir détailler avant le développement grâce à des diagrammes UML ou la rédaction de tâches écrites. Puis, la mise en pratique des plans proposés sur du développement de code pur. Je pense donc avoir été très à l'aise dans cette

partie purement de conception, partir de rien et arriver à un rendu utilisable, tandis qu'à l'opposé, j'ai eu plus de mal avec le fait de devoir tester et intégrer ce code.

Enfin, je pense avoir réussi à m'intégrer facilement dans le groupe. La communication, lors des réunions ou au moment voulu s'est très bien déroulée et j'aurais aimé pouvoir travailler un peu plus en équipe, car je pense à la suite de ce stage, que ce point est un réel point fort pour moi.

Mon ressenti

Finalement, pour avoir été sur un projet du début à la fin, j'ai pu voir certains points qui m'attiraient particulièrement. J'ai ainsi trouvé très intéressant de pouvoir proposer une architecture et de travailler avec l'UML. C'est en effet un point que j'ai grandement étudié à l'UTT et que j'appréciais alors déjà. Le fait de pouvoir appliquer cela et de proposer un design sur un projet réel m'a d'autant plus séduit.

La conception du logiciel (de son architecture), mais également de son format et de son rendu visuel m'ont donc particulièrement plu. Les détails de mon projet n'ayant pas été définis totalement à l'avance j'ai pu me pencher davantage sur la conception du composant, chose que j'ai fortement appréciée et pour laquelle je me suis trouvé assez efficace.

Pour finir, j'ai grandement aimé le fait de pouvoir travailler en équipe, assister à des réunions, communiquer dans le but de trouver une solution et essayer d'être force de proposition de ces solutions malgré le fait d'être stagiaire sur un sujet différent de l'équipe. Le travail en "Agile" était nouveau pour moi mais m'a beaucoup intéressé et j'ai d'ailleurs eu la chance d'avoir une formation à ce sujet.

Projet professionnel

Dès lors, pour ce qui est de mon parcours professionnel il convient alors de m'intéresser à deux points.

Le domaine : dès le départ le domaine du médical m'intéressait et il m'intéresse toujours autant, j'avais alors hésité à suivre une branche plus axée sur les biotechnologies. Mais je suis parfaitement satisfait de pouvoir allier le médical à l'informatique. Je pense d'ailleurs qu'avec l'intelligence artificielle par exemple, ces deux domaines ont un énorme potentiel pour l'avenir et je souhaiterais ainsi rester dans ce milieu. Je compte donc poursuivre plus tard dans le médical informatique, mais je reste conscient qu'il existe de nombreux autres domaines et que l'informatique en touchant la plupart.

Le rôle : J'ai beaucoup apprécié le rôle de développeur en Recherche et développement. Je pense être à l'aise dans le développement et que celui-ci me correspondrait bien. Cependant, j'aimerais pouvoir par la suite m'orienter plus sur de la gestion de projet logiciels, par exemple en tant qu'architecte logiciel aux vues de la partie conception que celui-ci présente et de la communication qu'il requiert lors des différentes réunions ou explications. De même, j'ai trouvé très passionnant les autres rôles que proposait la méthode Agile (Product Owner ou Scrum Master), définir un produit ou faire le lien dans une équipe sont deux responsabilités qui m'intéresseraient pour le futur.

Enfin, je souhaiterais par la suite rester dans le domaine médical sur des projets logiciels, tout en ayant plus de responsabilités et ainsi d'influence sur les choix pour le produit (design, visuel, stratégiques...), le tout, en étant proche d'une équipe. Je suis bien évidemment conscient que cela nécessiterait plus d'expérience et une bonne communication.

Discussion et éthique

Pour conclure, il est important de discuter du but de ce stage, voir même de mon projet professionnel et de l'impact que ceux-ci auront pour autour de moi.

- Pour ce qui est de mon stage, j'ai été bien sûr amené à me poser la question du but de du composant développé durant mon stage. Il est utilisé par les cliniques pour analyser les images non acceptées et en comprendre les raisons. Il arrive que ces raisons soient des erreurs humaines, ce à quoi la clinique pourrait proposer aux technologistes des formations pour ne pas reproduire ces erreurs. On peut alors être amené à se demander si les cliniques ne privilégieront pas de remplacer la personne responsable de trop d'erreurs, et donc de licencier celle-ci, s'interrogeant ainsi sur le but réel de cette analyse et son éthique.
- De même, souhaitant travailler plus tard dans l'industrie du médical en informatique, il est important de souligner que comme son nom l'indique ce domaine est industriel. Il est donc concevable de se questionner quant au lien entre l'économie dans l'industrie avec le milieu de la santé, et de s'intéresser aux questions d'éthiques que le coût de la santé peut parfois représenter. Il est cependant important de rappeler que les personnes travaillant dans ce milieu le font beaucoup pour améliorer la santé des autres et que celle-ci aura toujours un coût.

Conclusion

Dans le but d'analyser les acquisitions non-acceptées durant des examens de mammographie, je me suis penché durant mon stage sur la conception d'un micro-service d'analyse d'images répétées et rejetées. Ce service sera ainsi l'occasion pour les cliniques de comprendre les raisons des acquisitions ratées et pouvoir former les technologistes en conséquence. J'ai évolué pour cela au sein de l'équipe logiciel SwApps de General Electric Healthcare.

Dès lors, j'ai commencé par concevoir le micro-service, sa ressource utilisée et ses composants : application, interface et acceptance. Celui-ci faisait office de serveur et réalisait la gestion d'une ressource RRAEvent qu'il stockait sur une base de données. Puis, j'ai développé une interface utilisateur capable de récupérer les données du serveur, afin de produire des résultats d'analyse des images non-acceptés sous forme visuelle et facilement utilisable. De même, j'ai conçu une Gateway capable de faire le lien entre le serveur et la station d'acquisition du mammographe. Pour finir, il a fallu modifier cette station d'acquisition afin de créer l'objet RRAEvent, correspondant aux informations nécessaires à l'analyse des acquisitions non-acceptées.

Enfin, ce stage a été pour moi l'occasion d'appliquer ce que j'avais étudié, de voir certains points sous un œil différent et d'apprendre d'une manière différente de l'apprentissage scolaire. Cela m'a fait comprendre beaucoup de choses quant aux habitudes dans le développement et aux méthodes telles qu'Agile. De plus, j'ai eu la chance d'utiliser de nombreuses technologies d'avenir, ce qui pour une première expérience, représente une vraie chance par rapport à mon avenir professionnel.

Finalement, je souhaiterais intégrer plus tard le domaine du médical, ce qui a été pour moi l'occasion de faire un premier pas dans ce milieu. Cela m'a également permis d'observer les différents métiers qui y étaient liés et d'observer le fonctionnement des équipes.

Bibliographie

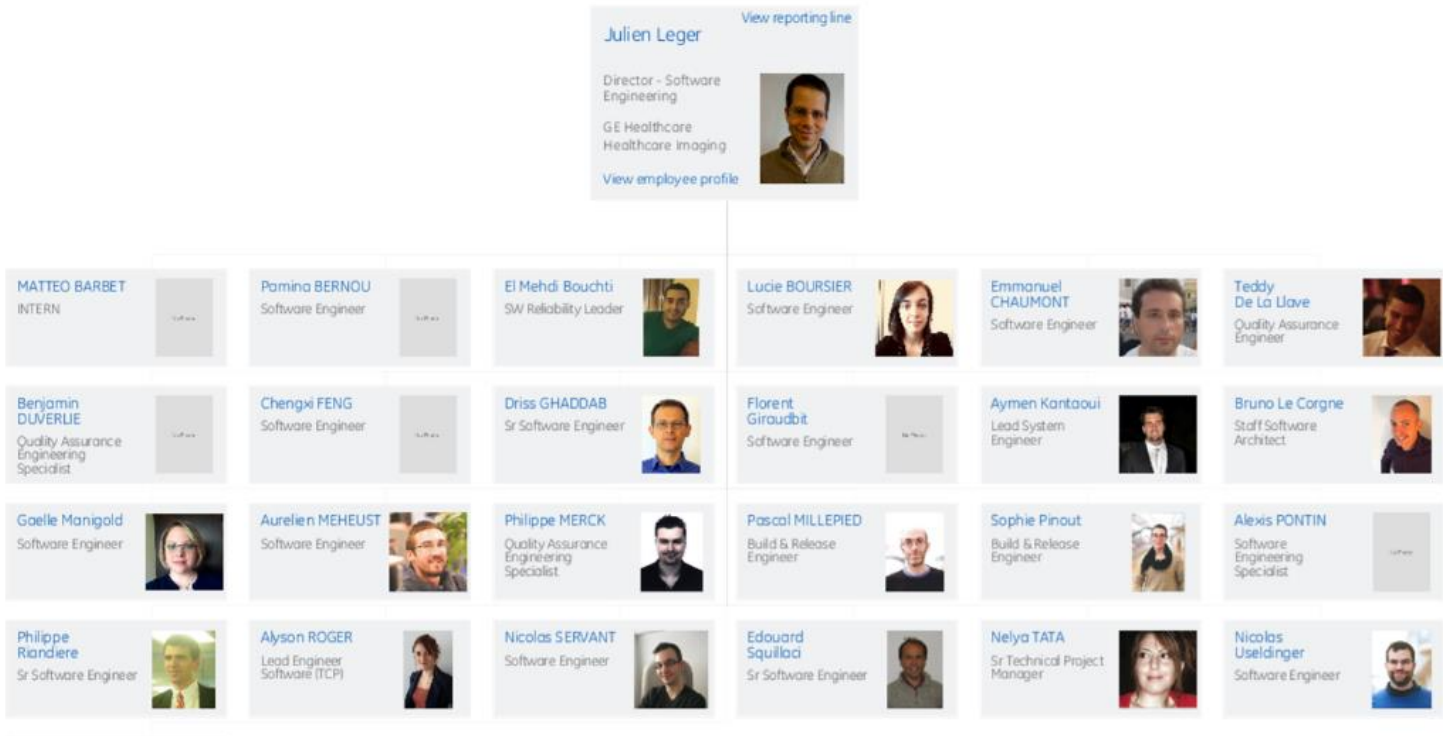
- ❖ *Apigee, (ebookGoogle cloud), Web API Design: The Missing Link. Dernière mise à jour le 17/06/2019, consulté le 22/08/2019. Dernière mise à jour en 2018, consulté le 28/07/2019. Disponible sur <https://cloud.google.com/files/apigee/apigee-web-api-design-the-missing-link-ebook.pdf>*
- ❖ *GE Healthcare, A propos de GE HealthCare. Dernière mise à jour le 18/03/2019, consulté le 04/09/2019. Disponible sur http://www3.gehealthcare.fr/fr-fr/a_propos_de_ge_healthcare*
- ❖ *GE Healthcare. Introducing the new General Electric. Dernière mise à jour le 24/08/2019, consulté le 21/09/2019. Disponible sur <https://www.gehealthcare.com/>*
- ❖ *GE Healthcare, Pristina Dueta. Dernière modification le 25/06/2019 consulté le 02/10/2019. Disponible sur <https://www.gehealthcare.com/products/mammography/pristina-dueta>*
- ❖ *Institut National du Cancer, Le cancer du sein. Dernière modification le 03/07/2019, consulté le 05/06/2019. Disponible sur <https://www.e-cancer.fr/Professionnels-de-sante/Les-chiffres-du-cancer-en-France/Epidemiologies-cancers/Les-cancers-les-plus-frequents/Cancer-du-sein>*
- ❖ *NEUMANN Idriss, Présentation des méthodes agiles et scrum. Dernière mise à jour le 17/06/2019, consulté le 22/08/2019. Disponible sur https://ineumann.developpez.com/tutoriels/alm/agile_scrum/*
- ❖ *Pages confluences de documentation de General Electric (bibliographie privée).*

Table des illustrations

FIGURE 1 : THOMAS EDISON, 1920	5
FIGURE 2 : GENERAL ELECTRIC EN FRANCE	5
FIGURE 3 : MAMMOGRAPHIE AVEC PRISTINA	8
FIGURE 4 : PRISTINA ET SA STATION D'ACQUISITION	9
FIGURE 5 : BÊTE À CORNES DU PROJET	10
FIGURE 6 : VUE DE RRA SUR LA STATION D'ACQUISITION DU MAMMOGRAPHE	10
FIGURE 7 : EXEMPLE D'ANALYSE DES IMAGES REJETÉES ET REPETÉES (SUR 2 ÉCHANTILLONS ACCEPTÉS)	11
FIGURE 8 : ITERATION SELON LA MÉTHODOLOGIE AGILE SCRUM	13
FIGURE 9 ET 10 : SCHEMAS DES COMPOSANT SERVEUR-GATEWAY-MAMMOGRAPHE	18
FIGURE 11 : SCHEMAS DU COMPOSANT DE L'INTERFACE UTILISATEUR D'ANALYSE	19
FIGURE 12 : PAGE HTML REPEAT REJECT ANALYSIS (FILTRE DE L'ANALYSE)	19
FIGURE 13 : STRUCTURE GÉNÉRALE D'UN MICRO-SERVICE DANS GIT	20
FIGURE 14 : RÔLE DES IDL ET DE DDSDATABASES	23
FIGURE 15 : SCHEMAS DES COMPOSANT UTILISÉS DANS LE MAMMOGRAPHE	24
FIGURE 16 ; SCHEMAS DE LA PAGE HTML SIMPLIFIÉE	24
FIGURE 17 ; APERÇU D'UN RÉSULTAT D'ANALYSE RRA	26
FIGURE 18 : TECHNOLOGIES UTILISÉES	29
FIGURE 19 : CAPTURES D'ÉCRAN DES RÉSULTATS SUR ACQUISITIONVIEWER	36
FIGURE 20 : CAPTURE D'ÉCRAN DU MICRO-SERVICE	37
FIGURE 21 : RÉSULTAT GRAPHIQUE CLIQUABLE D'UNE ANALYSE	39

Annexe

Organigramme de l'équipe SwApps



Tom McGuinness
President & CEO, GE Healthcare Imaging

Jay Hill
CTO/COO - Imaging

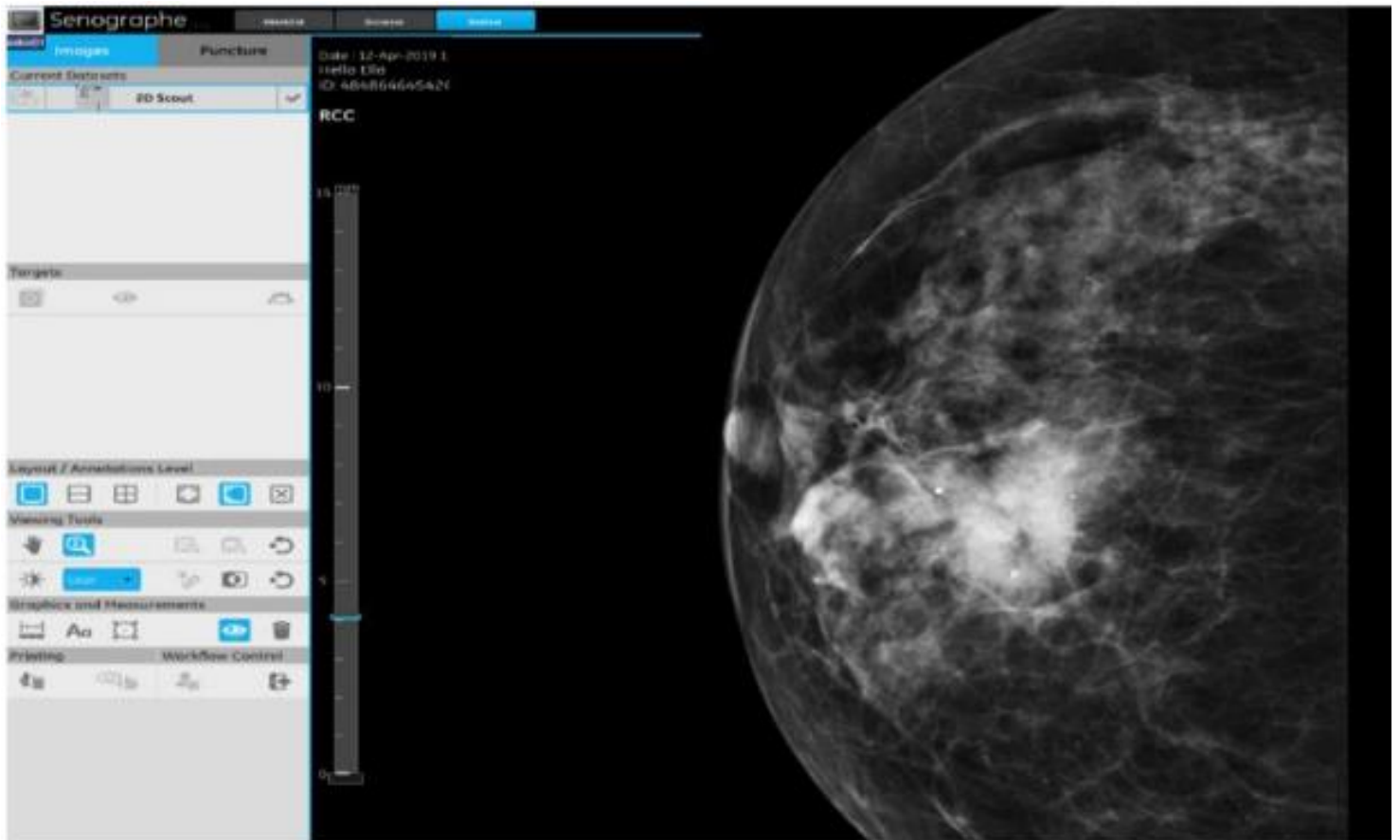
Jerome Gonichon
Sr Director - Software Engineering

Julien Leger
Director - Software Engineering

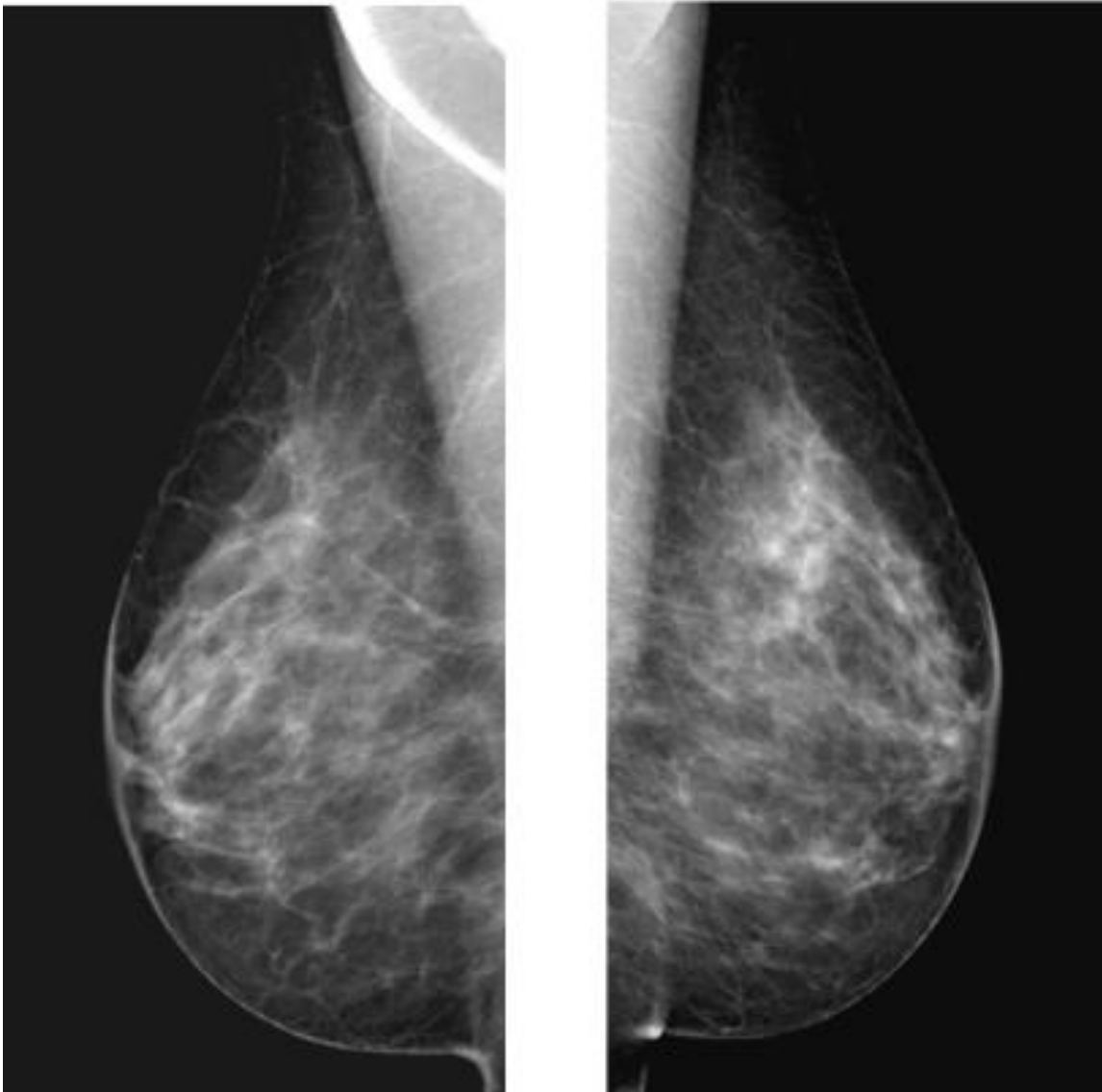
Organigramme direct chez GE Healthcare

MATTEO BARBET
INTERN
GE Healthcare Healthcare Imaging
[View employee profile](#)

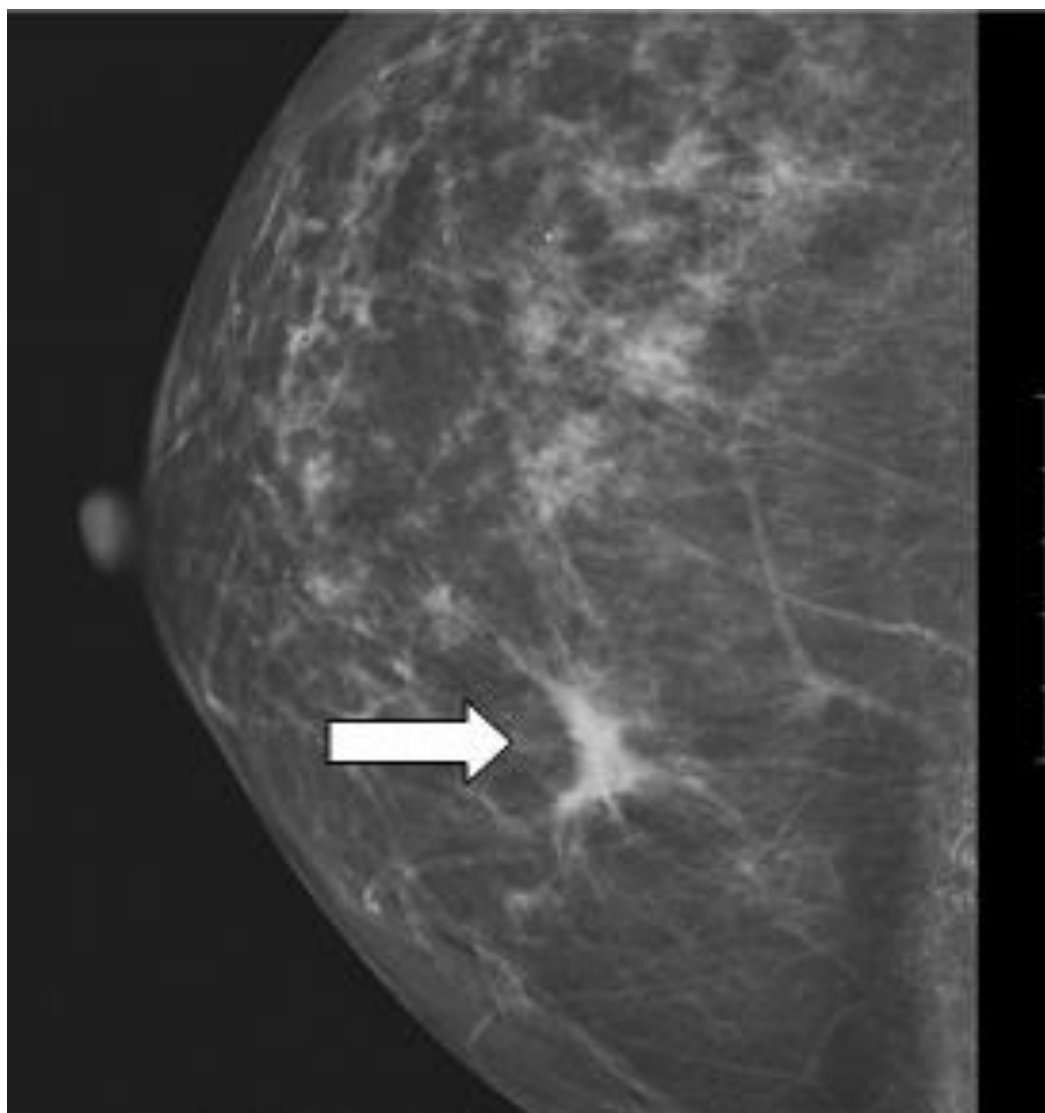
Capture d'écran du logiciel de la station d'Acquisition



Vue d'un mammogramme sein gauche et droit



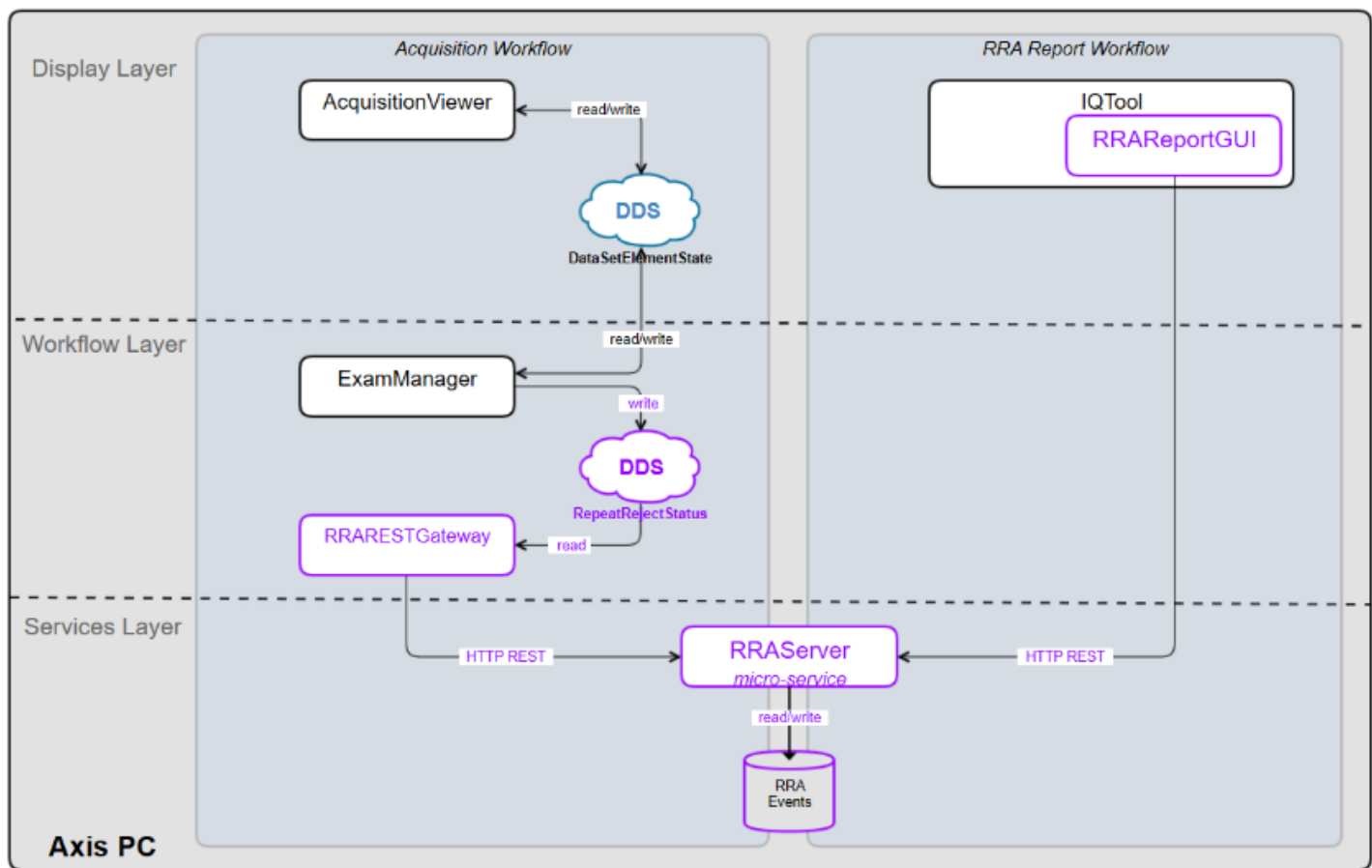
*Cas simple de détection de cancer du sein sur un
mammogramme*



Schémas des composants utilisés durant le stage

Components Involved in RRA

Legend:
 - purple => new component/interaction created for RRA
 - black => component/interaction already existing



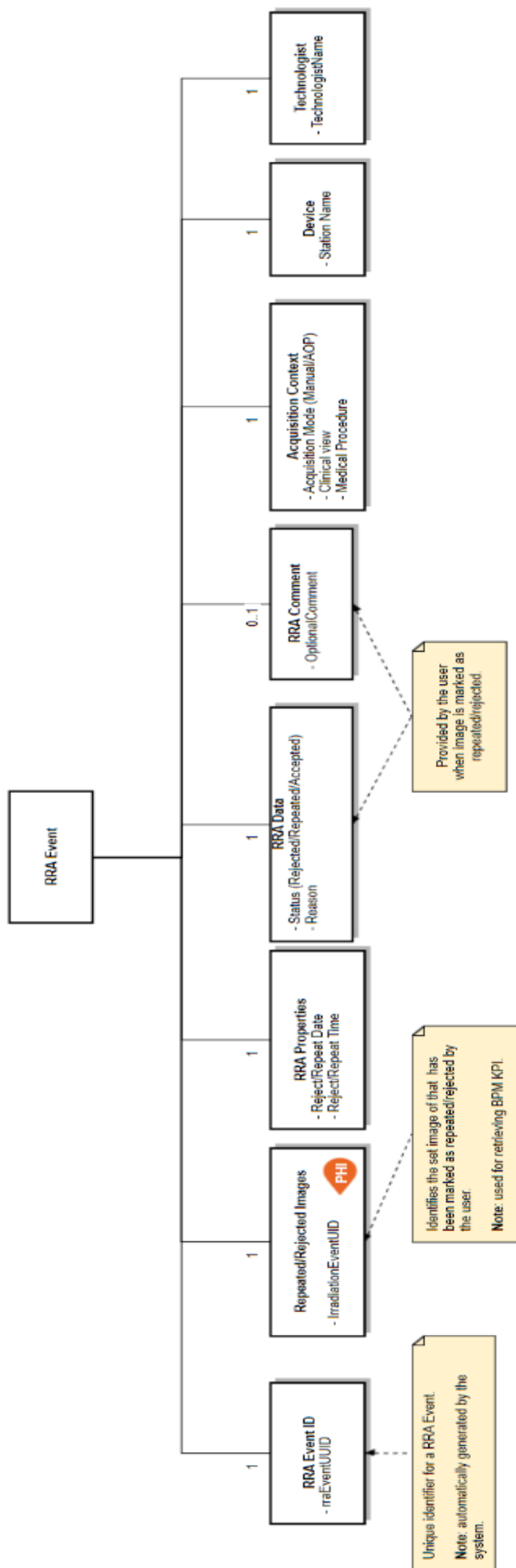
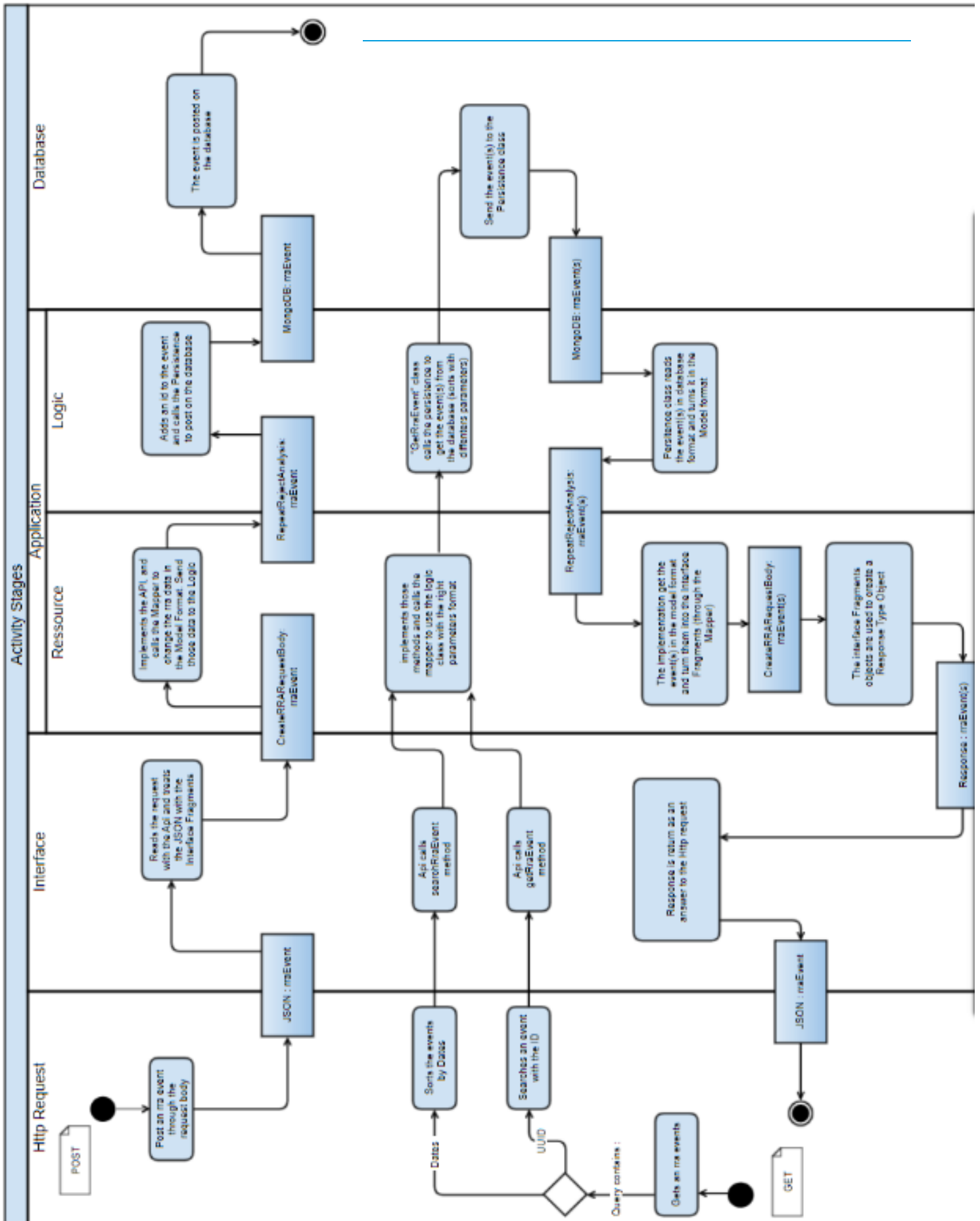


Diagramme d'objet d'un RRAEvent

Diagramme d'état transition d'un RRAEvent (au Post et au Get)



Filter the events by :

Period*

Start date:

08/10/2019 13:36

End date:

08/11/2019 13:36

* : required fields

Names

Device :

▼

Technologist :

▼

Group by*

Acquisition mode

Medical Procedure

Reason

Technologist

Show report

Reset

Capture d'écran de l'interface Utilisateur



Accepted : 100%
Rejected : 0%
Repeated : 0%

Reason	Accepted	Repeated	Rejected	Non accepted percentage
BLANK	2	0	0	0%
CLINICAL_ARTIFACTS	0	0	0	0%
EQUIPMENT_ARTIFACTS	0	0	0	0%
EQUIPMENT_FAILURE	0	0	0	0%
IMPROPER_EXPOSURE	0	0	0	0%
INTERVENTIONAL	0	0	0	0%
NON_CLINICAL	0	0	0	0%
NONE	0	0	0	0%
OTHER	0	0	0	0%
PATIENT_MOTION	0	0	0	0%
POSITION	0	0	0	0%
POOR_COMPRESSION	0	0	0	0%
WRONG_VIEWNAME	0	0	0	0%
Total	2	0	0	Non accepted: 0%

API du micro service RRA codé dans Swagger Editor (YAML)

Résultat du script "Check" ouvert via SonarQubes

